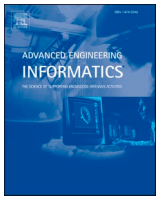
Advanced Engineering Informatics 69 (2026) 103995



Contents lists available at ScienceDirect

Advanced Engineering Informatics

journal homepage: www.elsevier.com/locate/aei



Full length article



Knowledge-driven automated design of industrial robots: A unified graph-based framework with multi-engine reasoning

Tao Sun, Bo Wang, Xinming Huo *

Key Laboratory of Mechanism Theory and Equipment Design of Ministry of Education, Tianjin University, Tianjin 300354, Republic of China

ARTICLE INFO

Keywords:
Robotic intelligent design
Graph-based representation
Knowledge reasoning
CBR and RBR

ABSTRACT

Industrial robot design is crucial for modern manufacturing but faces challenges in managing complex design information and knowledge, including topological features, motion/force specifications, non-parameterized component instances, and assembly rules. This study proposes an intelligent design framework for industrial robots that combines unified data representation with knowledge reasoning. Firstly, four type graphs are defined to store and transfer information throughout the design process. Subsequently, a motion/force analysis engine based on fusion rule-based reasoning (RBR) and case-based reasoning (CBR) is constructed to calculate mobility, design actuations and analyze constraints. Then, a component recommendation engine is built on knowledge graph (KG) with embedded domain expertise to suggest component instances based on motion/force analysis results and design requirements. Finally, an automated assembly of component CAD models with interface specifications is performed. The proposed framework is implemented as an intelligent design system for industrial robots. To validate the methodology, a machining parallel robot is designed subject to different requirements. Results demonstrate that our framework efficiently automates the transition from conceptual specifications to practical CAD models while meeting diverse design objectives, showing significant improvements in design efficiency compared to traditional methods.

1. Introduction

Industrial robots are extensively applied across a variety of fields, including manufacturing, aerospace, and logistics. They perform a wide range of operations such as machining, welding, pick-and-place tasks, and assembly [1–3]. To meet the specific demands of these diverse applications, the design of industrial robots focuses on understanding the relationship between mechanisms and their performance. However, the increasing complexity of industrial applications demands more efficient and automated design approaches to reduce development time and costs.

Generally speaking, robotic mechanisms are characterized by topological and dimensional parameters that are developed through extensive research efforts [4]. However, component design remains the most time-consuming and knowledge-intensive stage in this process. This challenge primarily arises from the diverse morphologies of joint components. They have significant impact on performance of robots yet resist straightforward parameterization due to the lack of standard basic forms. Additionally, components in robots, particularly in parallel and hybrid mechanisms, are interconnected based on specific rules [5,6]. These components not only have complex interfaces but also exhibit coupled motion and force characteristics. Although existing joint components from successful robot designs can serve as inspiration, they cannot be directly reused in new applications. Designers must not only account for desired performance metrics but also incorporate the inherent motion and force characteristics of the mechanism [7–9]. In most cases, joint shapes must be sketched from scratch based on experiential knowledge. As a result, the design process involves a fragmented body of knowledge that encompasses topological features, motion/force analysis, and design instance. Although designers attempt to document their design experience, they often require manual updates and lack a unified representation method to inherit [10–12]. Consequently, an efficient organization and integration of this fragmented knowledge is essential for automatic industrial robot design.

In this paper, an intelligent design approach for industrial robots is studied based on knowledge representation, retrieval and reasoning. Two fundamental challenges need to be addressed in this research. The primary challenge lies in achieving a unified data representation framework. The design process inherently involves complex interactions

* Corresponding author at: Key Laboratory of Mechanism Theory and Equipment Design of Ministry of Education, Tianjin University, Tianjin 300354, Republic of China.

E-mail addresses: stao@tju.edu.cn (T. Sun), wangbo_hb200@163.com (B. Wang), xmhuo@tju.edu.cn (X. Huo).

https://doi.org/10.1016/j.aei.2025.103995
Received 15 January 2025; Received in revised form 20 August 2025; Accepted 15 October 2025
Available online 3 November 2025
1474-0346/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Nomenclature

RBR	Rule-based reasoning
CBR	Case-based reasoning
T-Graph	Topological graph
KG	Knowledge graph
IKG	Industrial knowledge graph
Dof	Degree-of-freedom
F	Fixed platform
M	Moving platform
R	Revolute pair
P	Prismatic pair
H	Helical pair
S	Spherical pair
C	Cylindrical pair
U	Universal joint
PL	Planar pair
$\boldsymbol{v}$	Set of vertices
$\boldsymbol{E}$	Set of edges
$\boldsymbol{F}_v$	Property matrix of vertices
$Sim(\boldsymbol{a},\boldsymbol{b})$	Similarity between two instance
Sim_l, Sim_r	Similarity between two nodes and two edges of graph

among multiple domains, including topological configuration, motion/force characteristics, component modeling, and assembly-related specifications. Although these elements are traditionally represented using distinct formalisms, they are deeply interdependent. For instance, the representation of topological features is primarily graphical and symbolic, while the outcomes of motion/force analyses are mathematical in nature. This representational disparity hinders effective information recording and knowledge transfer. The second challenge lies in mapping knowledge across different design domains to enable automation. Motion and force analysis serve as critical examples in industrial robot design [13,14]. While mathematical models can theoretically calculate constraint and driving forces, they often fall short in addressing real-world engineering complexities [15,16]. Additionally, the correlation between joint components and overall robot performance remains poorly understood. This knowledge gap hinders the direct adaptation of existing robot designs to new requirements. As a result, each new application demands substantial redesign efforts and extensive testing to achieve optimal performance [7,17,18,19].

To address these challenges and facilitate automated industrial robot design, this paper proposes an intelligent design method for industrial robot based on unified data representation and fusion knowledge reasoning. We have made the following two main contributions to this research.

(1) A novel graph-based unified representation framework effectively captures and integrates the complex relationships between topology, performance, component, and assembly information in robot design. This approach systematically addresses the challenge of interconnected design data, enabling efficient knowledge transfer across different domains while minimizing manual interventions.
(2) A comprehensive knowledge reasoning system comprises three specialized engines – motion/force analysis, component recommendation, and automated CAD generation. These engines coordinate fragmented knowledge through a combination of rule-based reasoning (RBR), case-based reasoning (CBR), along with domain expertise to assist designers in complex decision-making tasks.

The paper is organized as follows: Section 2 reviews related literature in robotic data representation and knowledge reasoning-based design. Section 3 describes the research methodology. Section 4 presents the development of an automated design system and discusses the results through case studies. Finally, Section 5 provides conclusions.

2. Related work

2.1. Robotic data representation

The automatic design of robotic mechanism faces significant challenges due to the inherent complexity of these systems. Robotic mechanisms contain a variety of linkages and joints, requiring extensive domain knowledge for effective design. This complexity has driven research into knowledge extraction and mining approaches [20]. One proposed solution is the use of industrial knowledge graphs (IKGs), which represent robot parameters, properties, and applications in a triple format. While IKGs can provide useful recommendations to designers, their data structure makes full design automation difficult.

In practice, robots are often depicted through geometric symbols, while their applications or properties are recorded as text. However, this makes it challenging for computers to process the information for analysis and calculations. Some researchers have attempted to convert the geometric symbols to text, but this requires associating them with complex rules [21]. Others have explored analytical expressions [22], but these fail to include all needed information in one framework. Converting between different description formats frequently leads to information loss, requiring manual intervention by designers.

This highlights the need for a more comprehensive approach to robotic data representation. Graph-based representations have gained attention as an analogy for robotic mechanism components. The use of graph-based robot feature representation can be traced back to 1964, when Freudenstein mapped robot connections to graph elements such as nodes and edges [23,24]. This introduced the concept of adjacency graphs for representing topological information. In recent years, expanded graph definitions have been proposed, where bars are described as nodes and the joints are associated with edges, focusing primarily on the joint type. These approaches have been successfully applied to the topology synthesis of both open-chain and multi-loop mechanisms [25,26]. However, they may not fully capture other important details, such as dimensions and actuations [27], which are essential for robotic design. To overcome these limitations, this paper introduces a novel graph representation that not only encapsulates topological structure but also integrates motion/constraint and assembly interfaces within a unified framework, enabling a more complete description of robots.

Graphs have emerged as a versatile framework for digitally encoding robotic information [28]. Their structure, consisting of vertices (entities) and edges (connections between entities), can naturally capture the topology and dimensions of robotic mechanisms. The vertices represent individual components, while the edges depict their physical or logical relationships. This paper aims to leverage this graph-based representation to develop a more comprehensive approach for robotic data modeling.

2.2. Knowledge reasoning in design

Recent years have witnessed advancements in knowledge driven design. It has effectively addressed challenges such as the rapid reuse of design knowledge, heavy reliance on expert experience, and decision-making bottlenecks [29–31]. Early knowledge engineering focused on addressing design problems of large-scale mechanical components. For example, Zhang et al. [32] implemented an automated product design method using semantic representation of engineering knowledge using OWL and SWRL and applied it to the design process of shafts. Qin et al. [33] proposed an intelligent design method based on knowledge engineering for flexible gears. However, these studies primarily dealt with

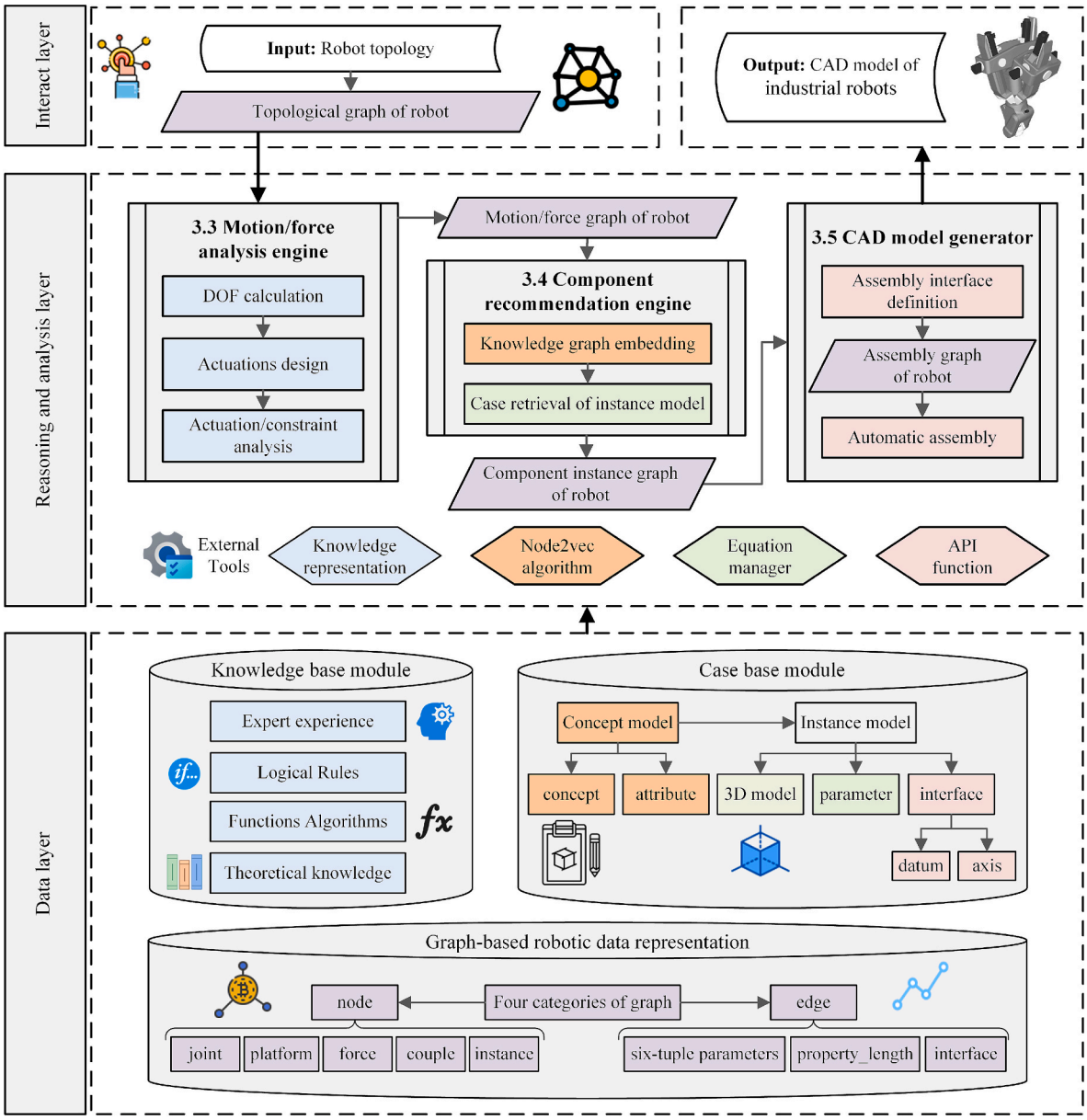


Fig. 1. The overall framework of the intelligent design system for industrial robots.

standardized components having simple structural forms.

To address the challenges of complex mechanical system, researchers have developed various knowledge-based reasoning methods. RBR has proven effective in scenarios with well-structured empirical knowledge, enabling rapid analysis based on specific requirements. Lin and Hsu [34] developed an RBR-based automated design system for stretching dies, which generates entity models through reasoning and analysis of user inputs, supported by an integrated knowledge base and database. Wang et al. [35] proposed a rule-driven product configuration method to optimize parameters and manage design complexity in mechanical product development. Similarly, Wei et al. [36] introduced a rule-based intelligent design approach for automotive interior structures, where design parameters and model modifications are determined by matching design requirements with established rules in a knowledge base.

CBR has demonstrated its effectiveness in design scenarios with extensive existing product cases, enabling new designs to build upon or adapt previous successful solutions rather than starting from scratch [37]. In this context, Huo et al. [38] integrated machine learning techniques with CBR to create an innovative approach for intelligent tool design. Their method retrieves similar cases and predicts tool parameters, advancing the intelligent development of mechanical products. In a different application, Ke et al. [39] developed a remanufacturing design method combining vector space models with CBR. Their approach processes user requirements and employs similarity algorithms to identify the most relevant cases, with its effectiveness validated through clutch design examples. Extending CBR to maintenance applications, Chen et al. [40] implemented a CBR-based fault diagnosis system for aero-engines, providing valuable decision support during maintenance operations.

While RBR offers structured reasoning capabilities, its static nature limits its application to individual mechanical products and fails to leverage existing successful design cases effectively. Similarly, CBR, which relies on attribute matching within a case database, becomes less effective when target case information is incomplete. To overcome these limitations, researchers have explored hybrid approaches combining RBR and CBR for enhanced automated and intelligent product design. For instance, Meng et al. [41] developed an integrated CBR-RBR method for rapid construction of parameterized bridge BIM models. Lon et al.

[42] addressed the challenge of reducing dependence on expert knowledge in mechanical product design by creating an automated design system that combines RBR and CBR, validating their approach through a corn sheller case study. Further advancing this hybrid approach, Peng et al. [43] integrated RBR with fuzzy comprehensive evaluation for determining fixture positioning schemes and features, while utilizing CBR to enable rapid customization of fixture conceptual designs.

In the development of knowledge-driven design, the application of technologies, whether individual RBR, CBR, or their fusion, requires definite and a limited number of empirical rules as a premise. However, for a large amount of uncertain prior knowledge, traditional reasoning methods cannot be used to complete the design tasks. Knowledge graph, as an important branch technology of artificial intelligence, can well represent structured and large-scale knowledge data. It uses graph structure to model and infer the complex relationships between things and has been widely used [44]. For example, Cheng et al. [45] constructed a multi-domain product design knowledge graph containing customer requirements, functional requirements, and design parameters and built mapping relationships between domains, which was successfully applied to the rapid design process of automotive products. Similarly, Akshay G. et al. [46] constructed a multi-relational and multi-level knowledge graph based on a large-scale 3D model dataset of product components, such as diverse bolts and nuts, which can effectively provide suggestions for product component reuse, automated design, and assembly. Guo et al. [47] used the knowledge graph to construct a product gene library containing information such as the structure, state, and related features of retired mechanical products, which can quickly and effectively automatically obtain part gene information and provide a data basis for the remanufacturing process.

Automating robotic mechanism design faces unique challenges in mapping between different domains. Motion and force analysis links topology with properties through mathematical models [13,14]. However, these models struggle to capture real-world complexities, particularly in systems with coupled motion and force interactions [15,16]. We also lack a clear understanding of how joint components affect overall robot performance. This makes it difficult to adapt existing designs to new requirements. Each new application thus requires extensive redesign work and testing to achieve good performance.

In the context of industrial robot component design, these challenges become particularly critical when developing 3D models for simulation analysis and topology validation. The design process involves many connected tasks. These include managing different types of data, using expert knowledge, applying theoretical models, and handling non-parametric component shapes. To solve these problems, we need to address two key challenges. First, we must develop a unified way to represent different types of design information. Second, we need better methods to transfer knowledge between design stages through integrated reasoning. This approach will help maintain complete information throughout the design process and create automatic connections between different design aspects.

3. Intelligent design system framework for industrial robot

3.1. Overview

Fig. 1 illustrates the technical roadmap of the proposed intelligent design system for industrial robots. This system employs graph-based design data representation, knowledge reasoning and parametric modeling, to efficiently design robotic structures and generate CAD models. It comprises three key layers: an interaction layer, an inference and analysis layer, and a data layer.

The interaction layer includes input and output modules. The input comprises design requirements and mechanism topology from the conceptual design stage. These design requirements specify the target application, such as high stiffness or large workspace. The output of system is detailed CAD models of the robotic structures.

The data layer is constructed to store the diverse data generated throughout the design process, from the initial requirements to the final CAD model. This includes topology information, motion/force analysis data, component details, and assembly information. While these data have varying formats, they are inherently interconnected. For instance, the component data is generated based on the topological structure of the mechanism. However, the topological description uses symbolic representations, while the component information is in the form of CAD models. This creates a gap between the different data types. To unify the management of this heterogeneous data, a novel unified method is proposed and will be introduced in Section 3.2. This method aims to seamlessly integrate the various data formats and their relationships, enabling efficient storage, retrieval, and utilization of the design information.

The inference and analysis layer is responsible for the crucial tasks of computation, analysis, and generation. Under the traditional design paradigm, design engineers must rely heavily on their own theory and experience to determine motion/force characteristics of the mechanism. Additionally, the structural design of individual components often involves reusing and modifying previous instances, due to a lack of robust theoretical support. A considerable amount of time and effort is spent managing the complex assembly constraint relationships between these components. To address these issues, this layer of the proposed intelligent design system incorporates three key execution engines:

(1) Motion/force Analysis Engine: This engine accepts a conceptual design topology as input and, through degree-of-freedom (DoF) computation and actuation/constraint analysis, results in the mechanism's force characteristics.
(2) Component Recommendation Engine: This engine retrieves and recommends suitable robot component instances from the component knowledge graph by performing similarity calculations based on extracted force information and application scenario requirements.
(3) CAD Model Generator: By defining the assembly interfaces of each component (including locating datum axes, locating datum planes, and rotating constraint datum planes), this generator automatically assembles the various components of the robot, rapidly constructing a complete three-dimensional model of the robot.

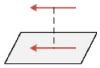
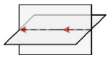
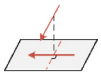
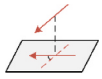
The design of industrial robot structures is a complex process that heavily relies on specialized kinematic knowledge and expert experience. This intelligent design system aims to reduce the need for such domain-specific expertise, significantly improving development efficiency and lowering design costs. Users of this system need only to input their design requirements and the mechanism topology into the interface. The performance analysis engine then employs rule-based and case-based reasoning methods to analyze the actuation and constraint characteristics of the mechanism. Next, a component recommendation engine retrieves suitable instances from the data layer, generating an optimized component design scheme. Finally, the CAD model generator automatically produces a complete CAD model with detailed structural features that meet the specified requirements. The resulting model can be used for further performance simulation analysis, providing users with a convenient and efficient design tool. Compared to traditional, experience-driven design methods, this system enables a more systematic and theoretically-grounded design process.

The above process outlines the overall technical route. The upcoming sections will provide detailed information regarding specific methods employed in the technical route.

3.2. Graph-based robotic data representation

The robotic design process aims to establish a mapping between the

Table 1
Geometric properties of two adjacent joint axes.

Relationship	Parallel	Collinear	Vertical	Orthogonal	Intersect	Arbitrary
Diagram						
Angle	0	0	$\pi/2$	$\pi/2$	$0 \sim \pi/2$	$0 \sim \pi/2$
Intersect	×	✓	×	✓	✓	×

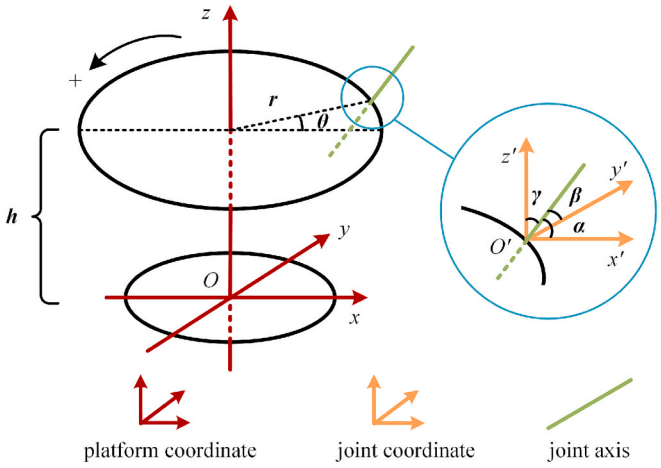


Fig. 2. Six-tuple parameters between joint and fixed/moving platform.

conceptual topology and the detailed structure, which encompasses diverse media data, such as topology, performance characteristics, component type and assembly conditions. They connect different design stages, but they exhibit inherent heterogeneity. Since they may be represented in text, symbols or images, engineers have to manually integrate them in the traditional industrial robot structure design and development process. To establish a comprehensive intelligent design system for industrial robot, the system is decomposed into multiple design engines. The output of the preceding engine serves as the input for the subsequent engine, ensuring the continuity and completeness of information flow between the engines. Therefore, it is necessary to use a unified data format to represent and store the information at each stage, facilitating effective communication and integration among the design engines.

From the perspective of information characteristics, the mechanism topology represents the types of joints and the relationships between them. The performance characteristics represent the types of forces and the relationships between motion and force. The component instance and assembly information represent the individual component instances and their assembly relationships. In essence, these diverse information elements can be described as a collection of entities and their relationships. To provide a unified representation of this information, a graph-based data structure is proposed. A graph could be defined as

$$\boldsymbol{G} = (\boldsymbol{V}, \boldsymbol{E}, \boldsymbol{F}_v, \boldsymbol{F}_e) \tag{1}$$

where $\boldsymbol{V} = \{v_1, \ldots, v_N\}$ is the set of vertices and $\boldsymbol{E} = \{e_1, \ldots, e_M\}$ is the set of edges, where each node and each edge are associated with a type. $\boldsymbol{F}_v \subseteq \mathbb{R}^{N \times p_v}$ denotes the property matrix of vertices. It is noted that the property of each vertex is described in p_v-D vector. $\boldsymbol{F}_e \subseteq \mathbb{R}^{N \times N \times p_e}$ is a 3-order tensor, with a p_e-D adjacent property vector axis.

The graph can effectively capture the various entities, such as joint types, force types, component instances, and their interconnections, enabling a comprehensive and integrated description of the robotic system. Four categories of graph are defined to represent the design data.

Definition 1. *Topological Graph (T-Graph). Two sets of vertices are defined, which denote bodies and joints, respectively. For the bodies, they generally include the fixed platform* (F) *and moving platform* (M). *The joint types of rigid robots generally involve revolute pair* (R *pair*), *prismatic pair* (P *pair*), *helical pair* (H *pair*). *Specifically, multi-DoF joints like spherical pair* (S *pair*), *cylindrical pair* (C *pair*), *Universal joint* (U *pair*), *and planar pair* (PL *pair*) *could be considered as a combination of several single-DoF joints. Edges in T-graph represent the adjacent relationship between the two nodes.*

The relationship of two adjacent joints is represented by "property_length", where "property" denotes the geometric properties classified into six types as shown in Table 1 and where "length" is the perpendicular distance between the axes.

The relationship between joint node and fixed/moving platform nodes is characterized by a six-tuple parameter as $\theta_r_h_\alpha_\beta_\gamma$. As shown in Fig. 2, θ_r_h describe the position of any point on the joint axis relative to the coordinate $O-xyz$ and α_β_γ are defined to describe the orientation of the joint axis.

Definition 2. *Motion/force Graph. It characterizes the actuation and constraint force of the mechanism. Based on T-Graph, novel nodes are added to represent actuated force, actuated couple, constraint force and constraint couple. These nodes are connected with joint nodes by edges, which indicate their relationship in the view of geometric properties. Based on Blanding's Law, the constraint force intersects* (*where parallelism is considered a special case of intersection*) *with the rotational motion axes, while it is perpendicular to translational motion axis. In this manner, the parallel, intersecting, and perpendicular spatial orientations from.*

Table 1 are used for description.

Definition 3. *Component instance Graph. It represents the mechanism configuration and visualizes the instance information of the recommended components. In this graph, nodes represent the instance types of each mechanism component, which are in the format of "Structure Type_Joint Category". For example,* "A_P" *denotes the instance of* P *joint with* A *Structure Type.*

Definition 4. *Assembly Graph. Based on the component instance graph, edges between two adjacent component instance nodes are equipped with assembly rules for interfaces. Based on the definition of assembly interfaces, edges are represented as "Interface Type_Interface Type", such as* "I_II". *It indicates that interface type* I *of the preceding component instance mates with interface type* II *of the subsequent component instance. The components are ordered by their distance from the fixed platform.*

These graphs are conducive to represent, record, update and compute the input and output information of each design stage. The relationship of the four kinds of graphs is presented in Fig. 3.

Based on the four definitions, we take a serial Scara robot as an example. SCARA robot is widely applied in picking and up, assembly, welding and so on. It is composite by RRPR, with all the motion axes are parallel with each other. As shown in Fig. 4, the data in terms of topology, motion/constraint, component instance and assembly conditions could be described by the four graphs. Compared with the example in Fig. 3, T-Graph of SCARA robot has no loop between the fix platform and moving platform. Therefore, from the T-Graph, the topology of both

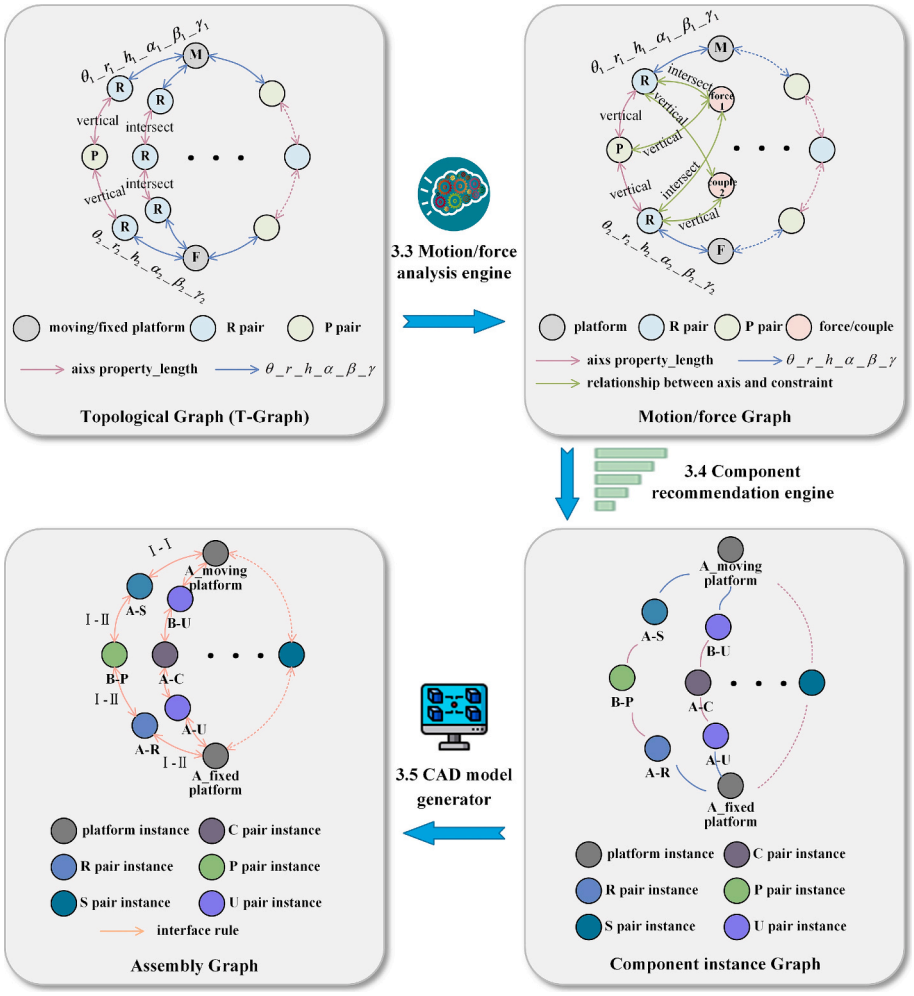


Fig. 3. Graph-based design data representation and its flow diagram.

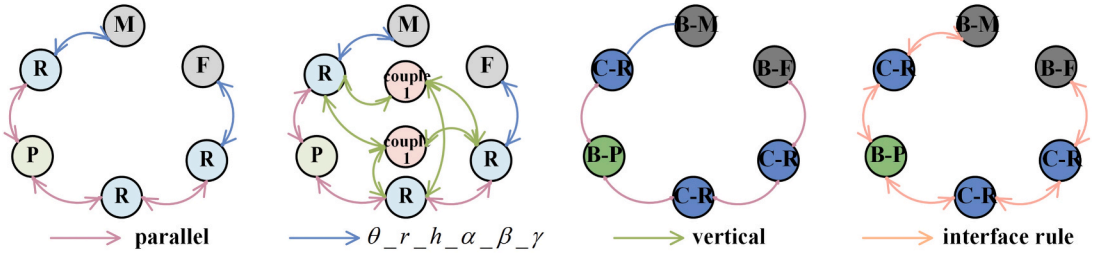


Fig. 4. Graph-based design data representation of a SCARA robot.

serial robot and multi-loop mechanisms would be recognized.

In the following sections, the methods for information transformation among graphs are introduced.

3.3. Motion/force analysis method based on RBR and CBR

In order to effectively obtain the motion and force of mechanisms with physical meanings, this section employs a RBR and CBR fusion approach. The rules represent the expert knowledge, structured in an "IF condition, THEN conclusion" format. The method proceeds through several sequential stages: initially, mechanism mobility is computed through the transformation of T-graph representations into Euclidean space coordinates. Subsequently, actuations are determined through rule-based inference. The final stage encompasses comprehensive actuation and constraint force analyses, utilizing a fusion approach that combines rule-based reasoning with cases. As a result, the motion/force graph is constructed, which serves as the foundational framework for component instance recommendation.

3.3.1. DOF calculation of industrial robot

DOF of a serial robot are simply equal to the number of joint nodes in its T-graph. However, the DOF calculation for parallel robots is significantly more complex due to the coupling between joints. Therefore, this section focuses on the DOF calculation for parallel robots.

The T-Graph presents joint types and relative axis orientations. Consequently, this paper employs screw theory to calculate the DOF. By inferring the axis characteristics (direction and position) of each joint, twists are constructed and used to calculate the mobility in three steps.

Table 2
Reasoning rules of joint axes.

Rules	IF	Then
RA1	Relationship of two axes = orthogonal	the former axes characteristics (position) = the next one
RA2	Relationship of two axes = parallel	the former axes characteristics (direction) = the next one
RA3	Relationship of two axes = collinear	the former axes characteristics = the next one
RA4	Relationship of two axes = vertical	the orthogonality of two axes characteristics (direction) = 0
RA5	Relationship of two axes = intersect	the orthogonality of two axes characteristics (direction) = 0 the position characteristics are equal
RA6	Relationship of two axes = arbitrary	the axes characteristics of two joints are retained
RA7	Joint type = R pair	the axes characteristics(position) are retained
RA8	Position of joint = initial | terminal	the axes characteristics are retained
RA9	Relationships of three axes = vertical | intersect	the middle axes characteristics⊥the former & latter one
RA10	Relationships of four axes = vertical & collinear	the middle axes characteristics⊥the former & latter one

Step 1: Limb Coordinate Transformation. The limb coordinate could be described by that of its first joint. According to T-Graph, the axes of the first and terminal joints could be obtained through homogeneous transformation. Taking coordinate at the fixed platform's center as the global reference frame, the position of the first joint axis is determined by parameters θ, r, and h, while its orientation is obtained from parameters α, β, and γ via a homogeneous transformation.

$$\boldsymbol{p} = (r\cos\theta\, r\sin\theta\, h) \tag{2}$$

$$\begin{aligned}\boldsymbol{T} &= \begin{pmatrix} \boldsymbol{R} & \boldsymbol{p} \\ \boldsymbol{0} & 1 \end{pmatrix},\ \boldsymbol{R} = \boldsymbol{R}_z(\gamma)\boldsymbol{R}_y(\beta)\boldsymbol{R}_x(\alpha) \\ &= \begin{bmatrix} \cos\gamma & -\sin\gamma & 0 \\ \sin\gamma & \cos\gamma & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha \\ 0 & \sin\alpha & \cos\alpha \end{bmatrix}\end{aligned} \tag{3}$$

where $\boldsymbol{T}$ is the 4*4 transformation matrix and $\boldsymbol{R}$ is a rotation matrix. "cos" and "sin" denote cosine and sine, respectively.

Step 2: Joint Coordinate Transformation. While the adjacent joint axes maintain known relative relationships, other joints require coordinate transformation analysis. To determine the precise orientation of joint axes, we establish a set of reasoning rules as defined in Table 2. Using the RBR approach, the characteristics of intermediate joint axes are derived through inference from the known parameters of initial and terminal joint axes.

Initial representation uses symbolic variables to define the direction and position of joint axes, with revolute joint positions marked as undetermined parameters. The orientation of axes is then determined through a dual-inference process. The first process applies forward inference, beginning from the first joint. The second process employs backward inference, starting from the terminal joint. The combined results of these inference processes are subsequently expressed as twists

$$\$_{t,i} = (\,\boldsymbol{s}_i \quad \boldsymbol{r}_i \times \boldsymbol{s}_i + h_i\boldsymbol{s}_i\,)^{\mathrm{T}} \tag{4}$$

herein, $\boldsymbol{s}_i$ and $\boldsymbol{r}_i$ denote the direction and position vectors of the ith motion axis. h_i is the pitch.

Step 3: DOF calculation. The motion twist system of the kinematic limb is formed by those of its n joints,

$$\$_t = (\,\$_{t,1} \quad \$_{t,2} \quad \cdots \quad \$_{t,n}\,) \tag{5}$$

The constraint wrench of the kinematic chain is derived through the reciprocal operation of the twists, as expressed in Eq. (6)

$$\$_{t,i}^{\mathrm{T}}\$_C = 0 \quad i = 1, 2, \cdots, n \tag{6}$$

The constraint force twist system of the mechanism is composed of the constraint force twists of all the kinematic limbs. The degree of freedom of the mechanism is reflected by the rank of the null space within this twist system. The detailed computational process and corresponding pseudocode are provided in the appendix.

3.3.2. Actuations design

In the context of industrial robots, the number of actuated joints is commonly equal to the mobility. Additionally, the configuration of these actuated joints should be optimized. If the robot operates in a serial configuration, each joint is actuated. This paper focuses on parallel mechanisms that meet two conditions: a) The mechanism consists of simple kinematic limbs with 2 to 6 degrees of freedom; and b) For actuated cylindrical joints, the translational direction is defined as the actuation direction.

The actuated joint selection rules (RBs) from reference [48–50] are extracted and summarized as follows:

1) IF a joint is the last joint in a kinematic chain, THEN it is not selected as an actuated joint;
2) IF a joint is of type S or U, THEN it is not selected as an actuated joint;
3) IF a joint is of type R and is not the first joint in a kinematic chain, THEN it is not selected as an actuated joint.

The automated process for actuated joints design is as follows: First, the joint types are extracted from the T-Graph and evaluated against the predefined rules for actuated joint selection. This filtering process identifies the joints that can be used as actuated joints. Next, all possible combinations of actuated joints within the mechanism are generated,

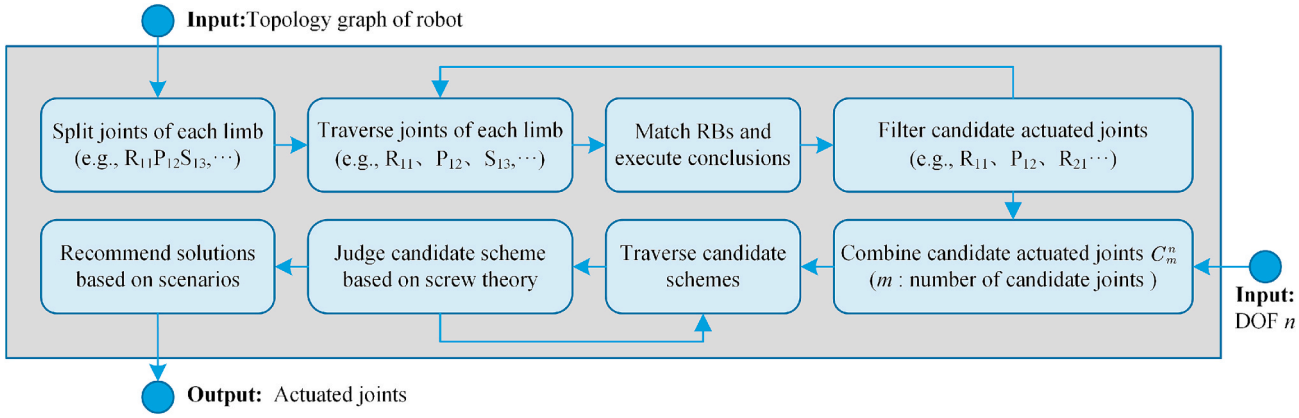


Fig. 5. Procedure for actuation design.

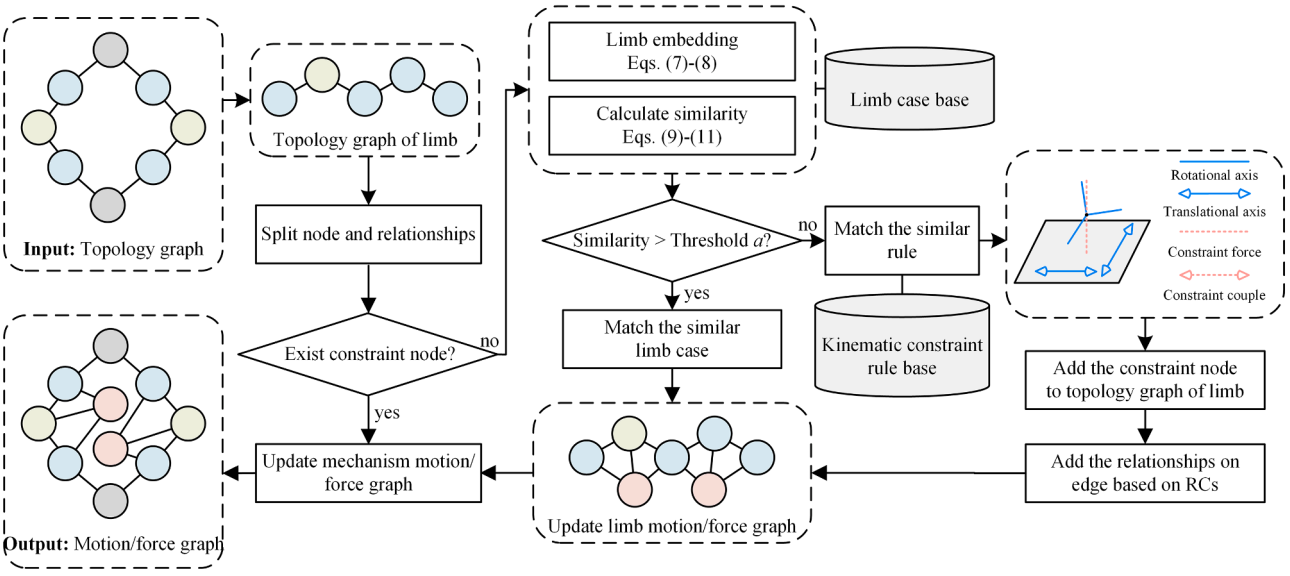


Fig. 6. Procedure for constraint analysis based on RBR and CBR.

Table 3
Relationship rules of constraint-joint node.

Rules	IF	Then
RC1	Joint node type = R pair & constraint node type = force	Relationship = intersect | parallel
RC2	Joint node type = P pair & constraint node type = force	Relationship = vertical
RC3	Joint node type = R pair & constraint node type = couple	Relationship = vertical
RC4	Joint node type = P pair & constraint node type = couple	Relationship = arbitrary

and the validity of each configuration is verified using screw theory [51]. Finally, suitable feasible solutions are recommended based on the specific application requirements. The process for configuring the mechanism's actuated joints is illustrated in Fig. 5. Once a feasible solution is determined, the T-Graph is updated to incorporate the actuated joint information.

3.3.3. Actuation/constraint analysis of mechanism

Component constraints play a key role in component design. In other words, component design should match its constraint in the mechanism, which can enhance the overall performance of the mechanism. To rapidly perform constraint analysis, this section proposes a RBR and CBR fusion approach. The process for constraint analysis base is illustrated in Fig. 6.

Based on Blinding's law, the kinematic chain constraints are determined using RBR method. A comprehensive kinematic constraint database has been developed (see [52]), containing 19 rules: 10 three-degree-of-freedom, 5 four-degree-of-freedom, and 4 five-degree-of-freedom kinematic chain constraints. This database enables the identification of constraint numbers and types within the kinematic chain. The T-Graph is then enhanced by adding constraint nodes, and the edge information is completed according to the constraint-joint node relationship rules outlined in Table 3, resulting in motion/force graph.

To improve efficiency, a case database of typical kinematic chain constraints has been developed using motion/force graphs. This database summarizes the force patterns and characteristics of common kinematic chains (UPS, UCU, PRPS, PRS, UPU, and RRRRR). It includes the direction and location of both driving and constraint forces within these chains under different actuation configurations.

The CBR method is used to search for matching patterns in the kinematic chain force database. When a similar case is identified, its force characteristics are applied to construct the corresponding motion/force graph.

Case retrieval requires appropriate case representation. The T-Graph contains discrete semantic information in its nodes and edges. Therefore, binary encoding is applied to process the node message. As illus-

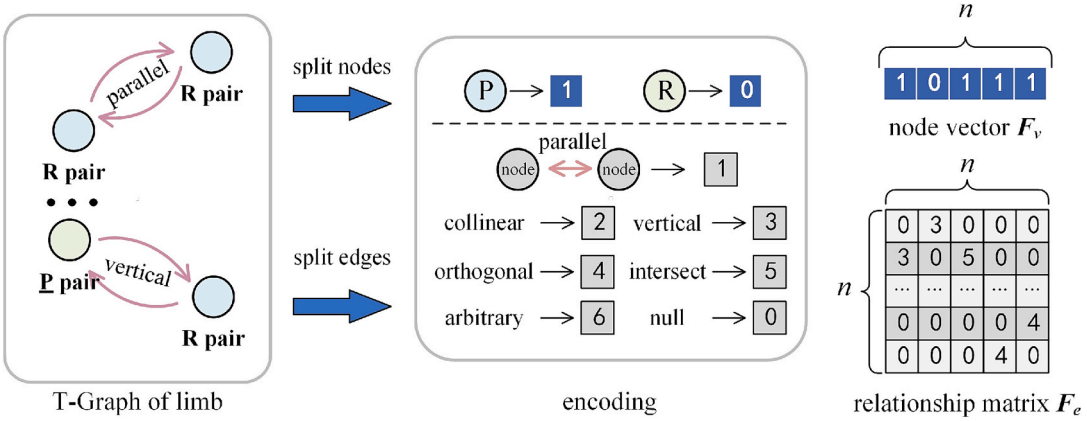


Fig. 7. Limb embeddings.

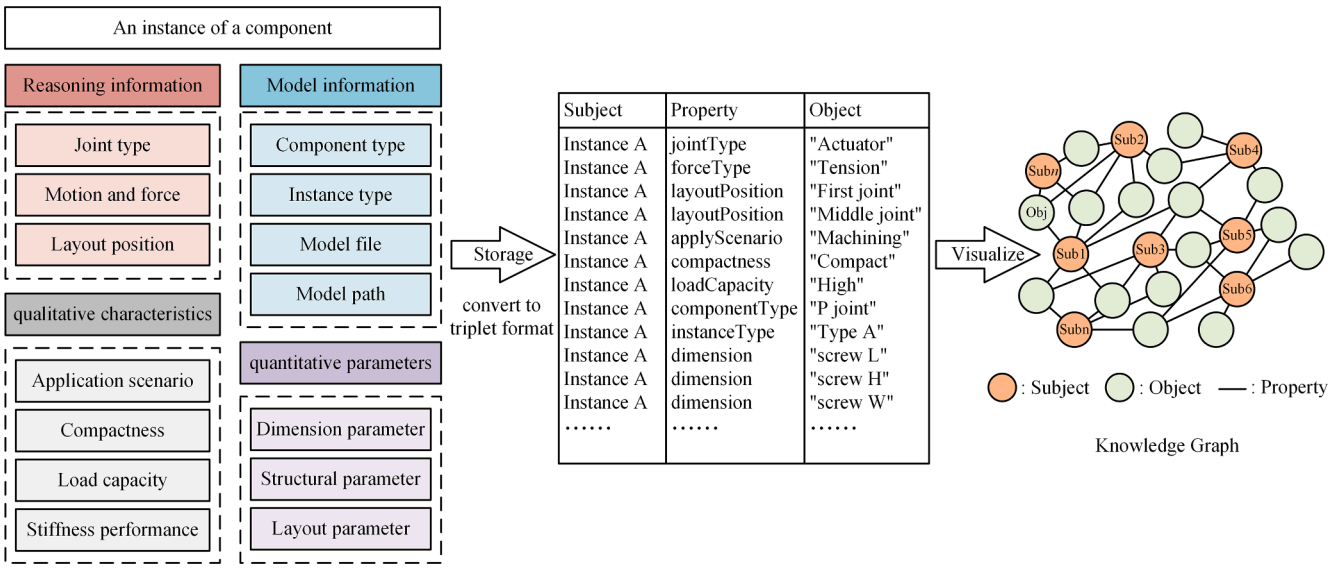


Fig. 8. Component Instance Library Construction Process.

trated in Fig. 7, joint nodes are encoded as follows: R joints are represented by "1" and P joints by "0". These encoded property of joint nodes are then transformed into a vector format according to Eq. (7).

$$\boldsymbol{F}_v = [j_1, j_2, \cdots j_n] \tag{7}$$

where $j_k (1 \leqslant k \leqslant n)$ is the Boolean value and indicates the type of each joint. The relationship between each node is described by 1–6 as in Fig. 7, and the property matrix of relationship (edge) is defined as

$$\boldsymbol{F}_e = \begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1(n)} \\ r_{21} & r_{22} & \cdots & r_{2(n)} \\ \cdots & \cdots & \ddots & \cdots \\ r_{(n)1} & r_{(n)2} & \cdots & r_{(n)(n)} \end{bmatrix}, \; r_{ik} = \begin{cases} 1 \sim 6 & \text{, adjacent nodes} \\ 0 & \text{, others} \end{cases} \tag{8}$$

where r_{ij} is the relationship between the ith node and the jth node.

Euclidean distance is applied to calculate the similarity between the encoded target and the source, respectively, and the case retrieval is completed by integrating the results. The calculation formulas are given in Eqs. (9)–(11).

$$Sim_l = 1 - \sqrt{\sum_{i=1}^{n} \left(j_i - j_{0,i}\right)^2}, 1 \leqslant i \leqslant n \tag{9}$$

$$Sim_r = 1 - \sqrt{\sum_{i=1}^{n} \sum_{k=1}^{n} \left(r_{ij} - r_{0,ik}\right)^2}, 1 \leqslant i \leqslant n, 1 \leqslant k \leqslant n \tag{10}$$

$$Sim = \omega_1 * Sim_l + \omega_2 * Sim_r, \; 0 \leqslant \omega_1 \leqslant 1, \; \omega_1 + \omega_2 = 1 \tag{11}$$

where Sim_l and Sim_r are the similarities of node and edge, respectively. j_i and $j_{0,i}$ denote the ith element in node vectors of the target and source limbs, respectively. r_{ik} and $r_{0,ik}$ represent the elements in the i-th row and k-th column of the relationship matrices of the target and source limbs, respectively. w_1 and w_2 indicate the importance of node and relationship similarity, respectively. Herein, $\omega_1 = \omega_2 = 0.5$.

3.4. Component retrieval method based on knowledge embedding

During the design process, engineers typically search for relevant component examples from previous successful designs and adapt them to meet current design requirements. In this section, a component

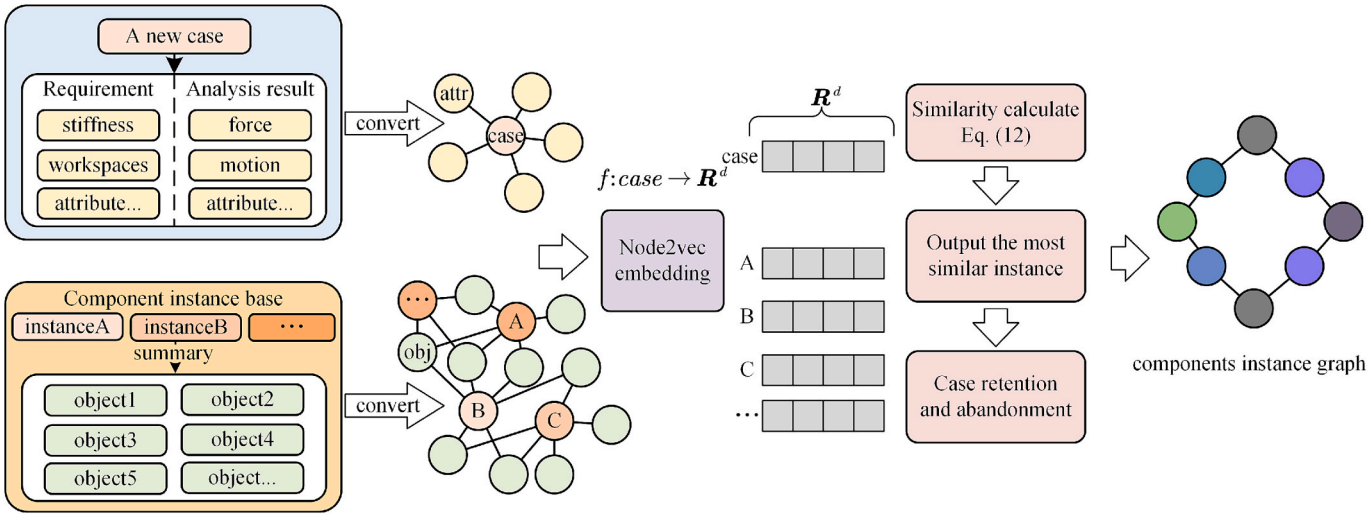


Fig. 9. Component Recommendation Engine Based Knowledge Graph.

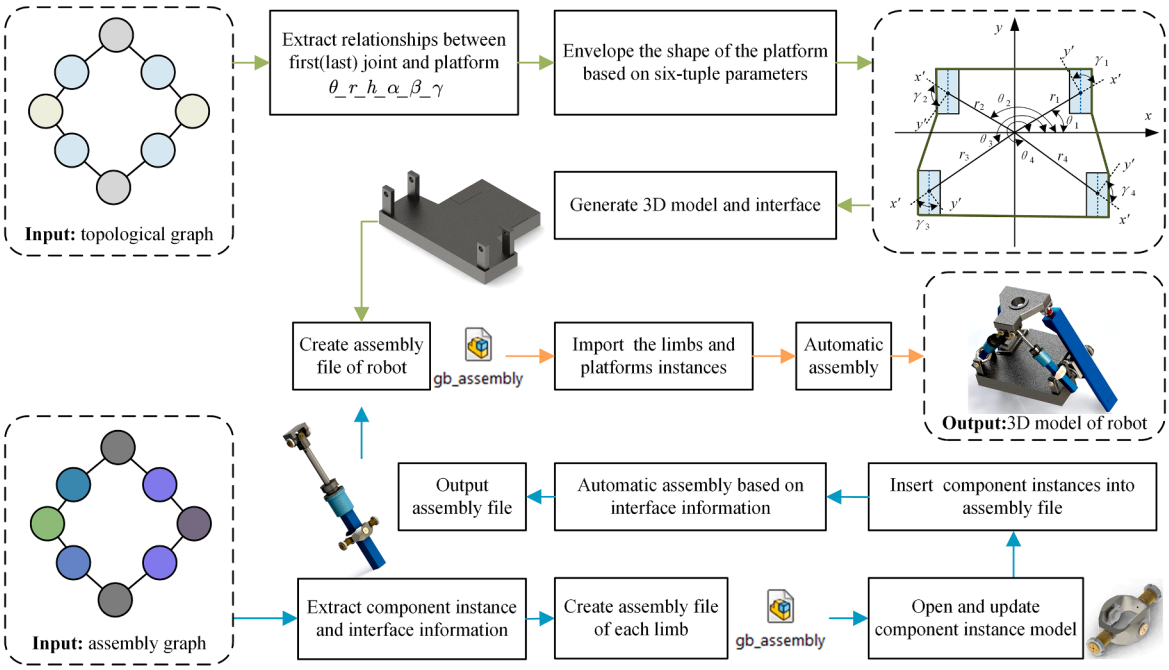


Fig. 10. Workflow of CAD model generation.

instance library has been developed to incorporate component attributes and expert knowledge.

A knowledge graph structure is adopted to build the component instance library, as it effectively represents and stores semantic attributes of component instances. Knowledge graphs provide a graphical organization of real-world entities and their relationships, offering machine-readable structured data that has found wide applications across various domains [45,53]. The instance library construction utilizes three key elements: entities, attributes, and relationships to represent component instances and their hierarchical connections. Parametric modeling techniques are applied to process these instances. As illustrated in Fig. 8, each component instance is characterized by four information categories: Model information, Reasoning information, Qualitative characteristics and quantitative parameters. These categories are converted into triples and stored in JSON format. The visualization is achieved through a 3D force-directed graph.

The workflow of the knowledge graph-based component recommendation engine is shown in Fig. 9. The first step involves creating a new component instance. This instance is constructed using the initial design requirements and the mechanism analysis results from Section 3.3, which include actuation and force characteristics. The construction follows the same data structure and representation format as the existing instance library.

The new instance is then integrated with the instance library data to form a network graph. A node2vec algorithm, based on random walk graph embedding, maps each node to a vector $\mathbb{R}^d$ in low-dimensional space. The training process involves adjusting node transition probabilities through appropriate parameter setting. This approach ensures the embedded vectors capture both homogeneous and structural similarities. The result is vector representations for both existing library instances and the new instance.

The final step involves calculating the similarity between existing and new component instances using cosine similarity. As shown in Eq. (12), this method measures the distance between their vector representations. A smaller distance value indicates greater similarity between two instances.

$$Sim(\boldsymbol{a},\boldsymbol{b}) = \frac{\boldsymbol{a}\cdot\boldsymbol{b}}{|\boldsymbol{a}|\times|\boldsymbol{b}|} = \frac{\sum_{i=1}^{n}(a_i \times b_i)}{\sqrt{\sum_{i=1}^{n} a_i^2} \times \sqrt{\sum_{i=1}^{n} b_i^2}} \tag{12}$$

herein, $\boldsymbol{a}$ and $\boldsymbol{b}$ are the embedding vectors of instant node and novel node with length n, respectively.

The system recommends the instance with the highest similarity score when the new component is compared to existing instances in the library. The recommended design is then processed through a 3D modeling software API for secondary development. This enables automatic updates of parametric component models based on different design parameters, generating multiple design alternatives.

As a result, a component instance graph is constructed based on the retrieval results and recommended instances, along with the parsed mechanism configuration. This graph guides the automated assembly process by calling the appropriate 3D component models to form kinematic chains and the complete mechanism.

3.5. Automatic CAD model assembly method driven by assembly graph

This study proposes a CAD model generation method based on assembly graph to rapidly create complete 3D mechanism models. Fig. 10 illustrates the automated assembly process. The CAD model generator creates new fixed and moving platform components based on information from the T-Graph, when the recommendation engine finds no similar instances in the library. For components with matching instances, their models are directly retrieved. The system then adds interface information to the component instance graph based on instance types, creating an assembly graph that guides the automated mechanism assembly.

The CAD model generator is developed using SolidWorks 2018 and VS2019, utilizing C# to call SolidWorks 2018 API functions to create and save documents, and manage mating relationships between models.

3.5.1. Assembly graph design

Two main approaches exist for automated assembly: coordinate transformation-based methods and feature-based assembly methods [54,55]. This study adopts the feature-based assembly approach to

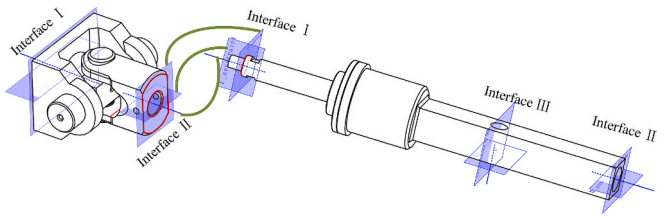


Fig. 11. Component Interface identification and Assembly.

enable post-assembly operations such as model manipulation.

The assembly interface features consist of one locating datum plane, one locating datum axis, and one rotation constraint datum plane. The locating datum plane and axis define the relative position between components, while the rotation constraint datum plane restricts their local degrees of freedom. Each component instance model has two or three sets of these assembly features. The number of assembly features for the fixed and moving platform components matches the number of kinematic chains in the mechanism.

The assembly process involves mating the corresponding interface features between components: locating datum axes through coaxial constraints, locating datum planes through coplanar constraints, and rotation constraint datum planes through angular constraints. As illustrated in Fig. 11, the component on the left contains two assembly interfaces (Interface I and Interface II), while the component on the right contains three interfaces (Interface I, Interface II, and Interface III). The assembly is performed by mating Interface II of the left component with Interface I of the right component.

The selection of assembly interfaces between two components is determined by their instance types. The system identifies these types from the component instance graph, then adds the corresponding interface features as edges to the graph. This process generates a complete mechanism assembly graph that guides the assembly sequence.

3.5.2. *Automatic assembly*

(1) Fixed and Moving Platform Generation

Fig. 10 illustrates the process of generating fixed and moving platform instances based on the T-Graph (indicated by green arrows). The process consists of three main steps. First, the system extracts the relationships between joint axes and platforms from the T-Graph, which define the arrangement of kinematic chains. Second, the system generates the platform shapes by creating envelope geometries based on the six-tuple parameters derived from these relationships. The component instances are then created through basic modeling operations such as extrusion. Finally, the system updates the assembly interface features on both platforms according to the types of the first and last joints in each kinematic chain.

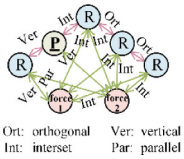
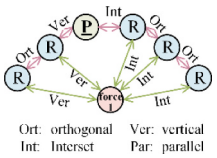
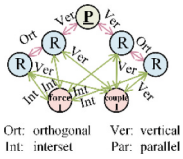
(2) Automated Kinematic Chain Assembly.

The automated assembly procedure for individual kinematic chains (shown by blue arrows in Fig. 10) follows a systematic approach. Initially, the system extracts the component instance models for each kinematic chain from the mechanism assembly graph and imports them into the chain assembly module. Next, the system executes the assembly operations according to the predefined interface information. The process then repeats for all kinematic chains in the mechanism until a complete assembly file is generated.

(3) Output of the Complete 3D Mechanism Model.

The generation of the complete 3D mechanism model (indicated by orange arrows in Fig. 10) follows a final integration phase. The system imports both the previously generated platform instances and kinematic chain instances into the complete mechanism assembly module. Based on the interface information, the system applies mating constraints between the kinematic chains and their corresponding platforms. The final step involves saving and exporting the fully assembled 3D mechanism model.

Table 4
Typical force analysis cases.

Limb topology	Topological graph	Limb code
RPS	 Ort: orthogonal Ver: vertical Int: interset Par: parallel	F_v 1 0 1 1 1 F_e 0 3 0 0 0 / 3 0 5 0 0 / 0 5 0 4 0 / 0 0 4 0 4 / 0 0 0 4 0
UPS	 Ort: orthogonal Ver: vertical Int: Interset Par: parallel	F_v 1 1 0 1 1 1 F_e 0 4 0 0 0 0 / 4 0 3 0 0 0 / 0 3 0 5 0 0 / 0 0 5 0 4 0 / 0 0 0 4 0 4 / 0 0 0 0 4 0
UPU	 Ort: orthogonal Ver: vertical Int: interset Par: parallel	F_v 1 1 0 1 1 F_e 0 4 0 0 0 / 4 0 3 0 0 / 0 3 0 3 0 / 0 0 3 0 4 / 0 0 0 4 0

4. Case study

Parallel machining robots constitute an important category of industrial robots. Their exceptional stiffness and dynamic performance characteristics have drawn significant research interest, resulting in their increasing adoption across multiple industrial applications [56,57]. However, the design of parallel machining robots presents considerable challenges due to the complex interactions between structural configuration and performance parameters. This complexity requires extensive non-parametric knowledge in the design process. This section presents a design system for parallel machining robots developed within the proposed intelligent design framework. This section presents a design system for machining robots developed within the proposed intelligent design framework.

4.1. *Design knowledge base for parallel machining robot*

The knowledge base for parallel machining robot design consists of two primary elements. The first element is a design rule base that contains descriptive knowledge and design rules. The second element is a design instance library that stores successful design examples, including their 3D models and associated attributes.

(1) Design rule base.

The design rule base incorporates domain expert knowledge through "IF-THEN" rule structures. These rules encompass several key components: joint axis inference rules (detailed in Table 2), degree of freedom calculation formulas based on screw theory (presented in Equations (4)–(6)), actuated joint selection rules (RBs 1–4), and constraint analysis rules (summarized in Table 3).

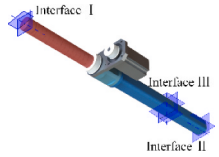
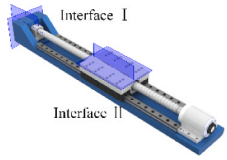
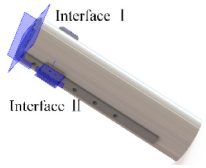
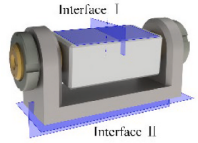
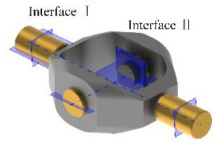
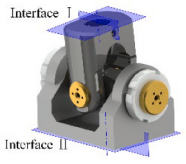
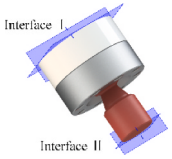
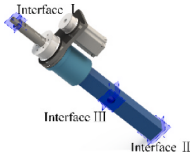
(2) Design instance library.

The design instance library is structured as a knowledge graph, which incorporates a typical constraint case base and a component instance base.

The constraint case base is derived from patent analysis and encompasses 50 common parallel kinematic machine chain configurations. For each configuration, the force state has been analyzed and graphically documented. The representation method described in Section 3.3.3 was utilized for processing these cases. Several examples of typical kinematic chain force cases are presented in Table 4.

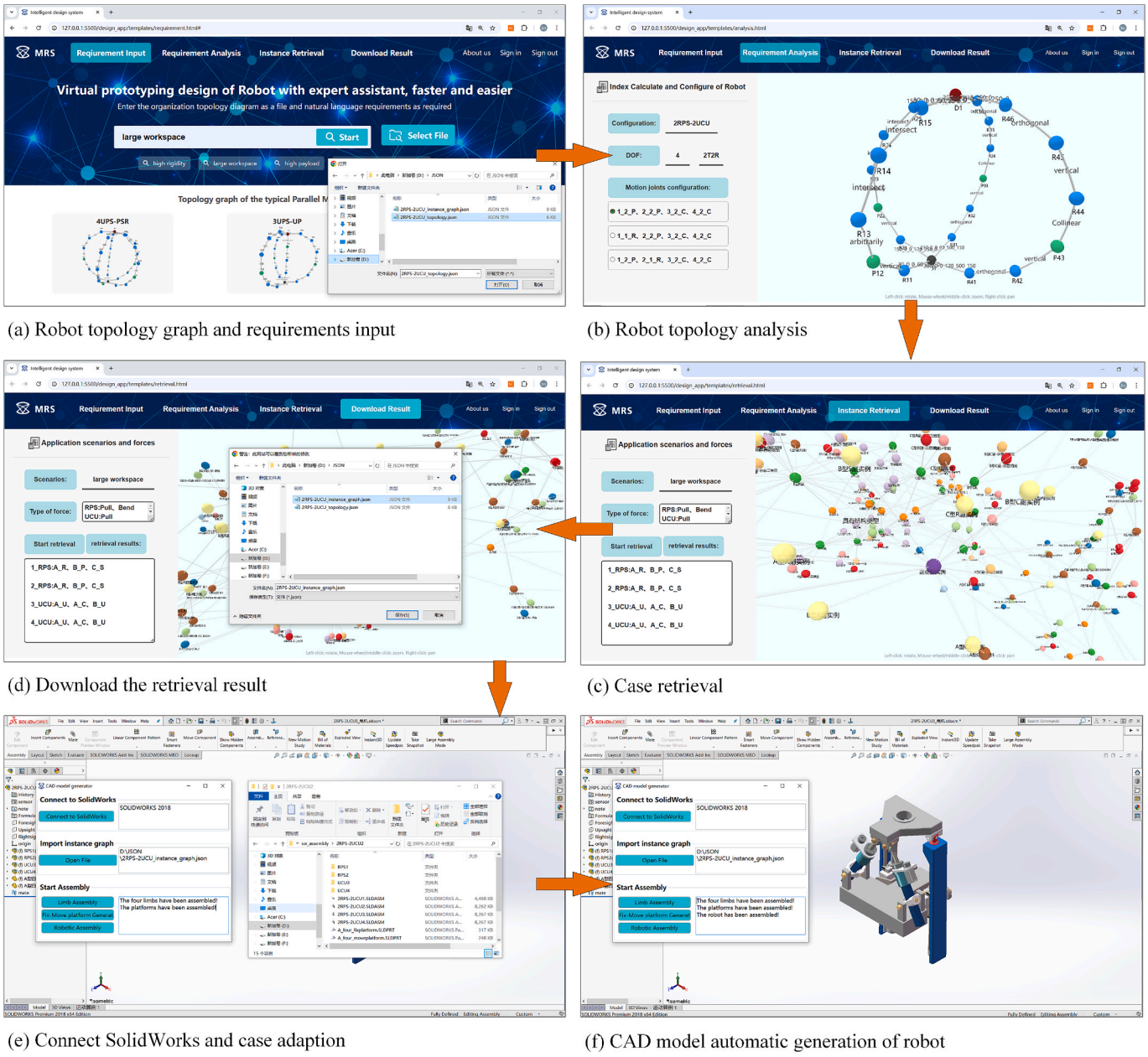
The component instance library contains conceptual models and multiple design variants for the following joint and link types: three types of P joints, two types of R joints, two types of U joints, three types

Table 5
Typical component instance.

Name	Model and interface	Joint type	Force type	Layout Position	Parameters
Type A P Pair		Actuator	Tension	Middle	Screw diameter Screw length
Type B P Pair		Actuator	Tension Bending	First Middle	Screw diameter Screw length Slider width Slider Span
Type C P Pair		Passive	Bending Torque	First Middle End	Slide rail length Slide rail width Slide rail height Cylinder diameter Cylinder length
Type A R Pair		Passive	Bending Torque	First Middle End	Sphere diameter Base length Base width Base height
Type A U Pair		Passive	Torque	First Middle	Sphere diameter Bracket length Bracket width Bracket height
Type B U Pair		Passive	Torque	First End	Sphere diameter Base length Base width Base height
Type A S Pair		Passive	Null	First Middle End	Sphere diameter Overall height
Type A C Pair		Actuator	Tension	Middle	Cylindrical link length Cylindrical link diameter

of S joints, two types of C joints, and three types of structural links. These component models are developed using SolidWorks parametric modeling techniques. Their characteristic properties and attributes are then thoroughly documented to complete the library. Key parameters reflecting the structural geometry of the components were selected, and the relationships between these dimensions are established. Datum features are defined to create standardized component model interfaces, with the specific definition standards detailed in Section 3.5.2. Table 5 provides instance information for several of the components in the library.

Due to the primarily qualitative nature of component instance parameters, the component instance library is represented and visualized using a knowledge graph structure. In this graph, nodes represent the individual entities, while edges define the relationships between these entities.



(a) Robot topology graph and requirements input

(b) Robot topology analysis

(c) Case retrieval

(d) Download the retrieval result

(e) Connect SolidWorks and case adaption

(f) CAD model automatic generation of robot

Fig. 12. Design case of parallel machining robot with 4-DoF.

4.2. Intelligent design system

An intelligent design system for parallel machining robots is developed based on the proposed framework and intelligent design methodology. The system was implemented using API functions from PyCharm 2022, Visual Studio 2019, and SolidWorks 2018. It automatically processes design requirements, determines the necessary information for each design stage, provides component recommendations, and generates complete mechanism CAD models.

The design process for a four-degree-of-freedom parallel machining robot, 2RPS-2UCU, is illustrated in Fig. 12.

The requirement input interface is illustrated in Fig. 12(a). Based on the graph structure definition rules described in Section 3.2, the system constructs a T-Graph which can be input through the interface. Design requirements are specified through a dedicated input field. To demonstrate the process, a four-degree-of-freedom parallel machining robot with 2RPS-2UCU topology is used as a case study. For this example, a T-Graph file is created and input into the system along with the design requirement specification of "large workspace".

During the motion/force analysis stage, the system renders and visualizes the mechanism topology input. As shown in Fig. 12(b), different node colors are used to distinguish joint and platform types. Specifically, R joints are represented by blue nodes, P joints by green nodes, while the moving platform and fixed platform are depicted by red and black nodes, respectively. The system analyzes the T-Graph to determine the mechanism configuration. By applying Eqs. (2)–(6) in combination with the established rule base, the system constructs joint motion screws and calculates the degrees of freedom. For the 2RPS-2UCU mechanism shown on the right side of the interface, the analysis reveals two rotational and two translational degrees of freedom.

The system provides recommendations for feasible actuated joints. For this mechanism, three actuation configurations are suggested: Configuration 1: "1_2_P, 2_2_P, 3_2_C, 4_2_C", Configuration 2: "1_1_R, 2_2_P, 3_2_C, 4_2_C", Configuration 3: "1_1_P, 2_2_R, 3_2_C, 4_2_C". In each configuration, the number of actuated joints matches the mechanism's mobility. The notation follows a three-part format: the first

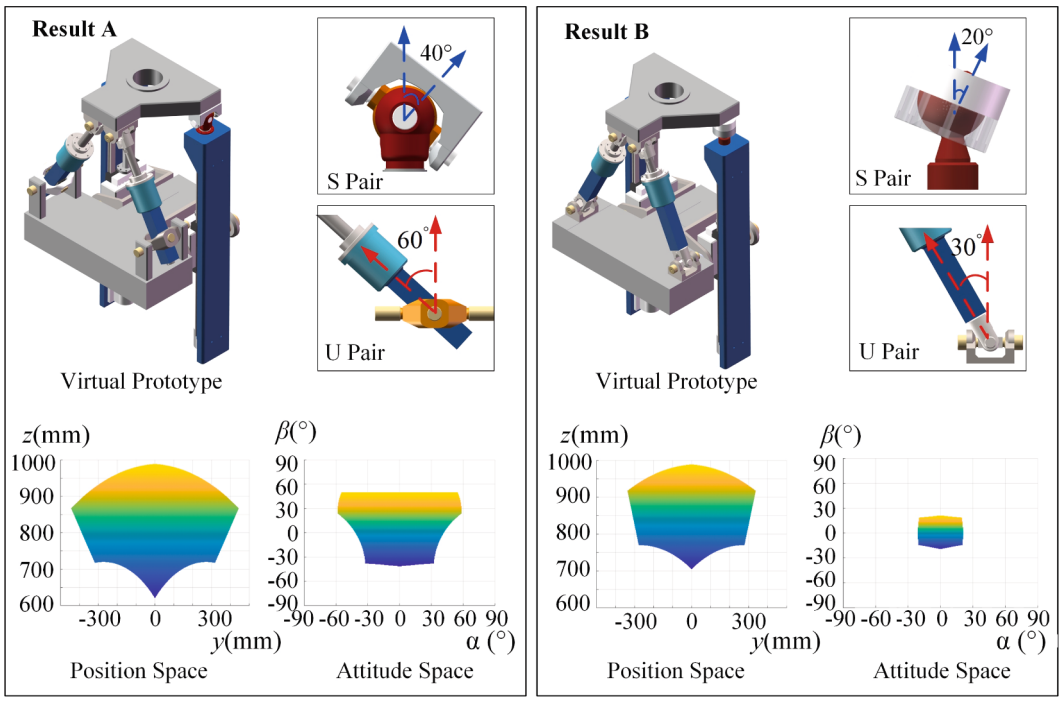


Fig. 13. Workspace analysis of virtual prototypes A and B.

number indicates the limb position index, the second number represents the joint position index within the limb, and the final letter denotes the actuation joint type. For the subsequent design process, Configuration 1 is selected.

Based on the recommended drive joint configuration scheme, the force characteristics of each kinematic chain are analyzed. Combining this with the design requirements, the component instance library is searched, and the results are displayed on the interface. Users can download the retrieved results as a component instance scheme diagram to their local machine. Fig. 12(c)–(d) shows the interface of the component instance recommendation engine.

The motion/force analysis of the 2RPS-2UCU mechanism displayed on the left side of the interface reveals distinct force characteristics for different limb types. The RPS limb experiences both pull/push forces and turning torque, while the UCU limb is subjected to pull/push forces only. Upon activating the "Start retrieval" button, the system generates component instance matches for each limb configuration. For the RPS limb, the system returns "1_RPS: A_R, B_P, C_S", indicating the following component matches: R joint: Type A with 68.2 % matching rate, P joint: Type B with 74.14 % matching rate, S joint: Type C with 75.88 % matching rate. For the UCU limb, the analysis yields: First U joint: Type A with 75.24 % matching rate, C joint: Type A with 74.32 % matching rate, Second U joint: Type B with 73.80 % matching rate.

The system initiates by verifying the connection to SolidWorks on the local machine. The component instance graph is transformed into an assembly graph by integrating individual component structural characteristics with established design principles. The system then executes the mechanism assembly process using data from both topology and assembly graphs, which includes kinematic chain assembly and platform generation. As illustrated in Fig. 12(e)–(f), the CAD model generation follows a systematic procedure by three steps: (1) The system establishes connection with SolidWorks. (2) The component instance graph is imported from the web database into the CAD generator. (3) The automated assembly sequence is executed through sequential operations: "Limb Assembly", "Fixed and Moving Platform Generation" and Complete Robotic Assembly". The resulting comprehensive mechanical model serves as a foundation for subsequent simulation studies and design optimization processes.

4.3. Discussion

4.3.1. Performance analysis of machining robot

The final mechanism configuration is fundamentally influenced by the specified design requirements. Subjecting to the "large workspace" requirement, the previous design (denoted as design A) incorporates a type A U-joint with hollow characteristic in the UCU chain to enhance both position and attitude workspace. In the RPS chain, the S-joint is implemented as a U-joint + R-joint instance, increasing rotational angle and further enlarging the translational displacement in the x direction.

As a comparison, we develop a second prototype (design B) of the 2RPS-2UCU parallel machining mechanism, with the input requirement as "high stiffness". Following the same design procedure described in Section 4.2, the model is generated as shown in Fig. 13(b). The structural configuration of mechanism B exhibits notable differences compared to the design A, reflecting their distinct design priorities. To meet the high stiffness requirement, mechanism B utilizes a solid U-joint in the UCU chain and a spherical S-joint in the RPS chain to improve overall stiffness, at the expense of pose capability.

A comprehensive performance comparison is conducted through two analytical approaches. ANSYS is utilized to simulate the overall stiffness of mechanism A and B, and a custom-developed MATLAB program is applied to compute and analyze the workspace based on the inverse kinematic model. The comparative analysis of these two mechanisms, with their distinct structural configurations and performance characteristics, validates the capability of this system to generate solutions tailored to specific performance requirements.

Workspace analysis indicates that both mechanisms exhibit translational motion in the y and z directions and rotational motion in the x and y directions. Design A exhibits 50.56 % positional workspace and 25.23 % orientational workspace, while mechanism B exhibits 33.68 % positional workspace and 5.51 % orientational workspace. Mechanism A demonstrates superior translational and rotational performance

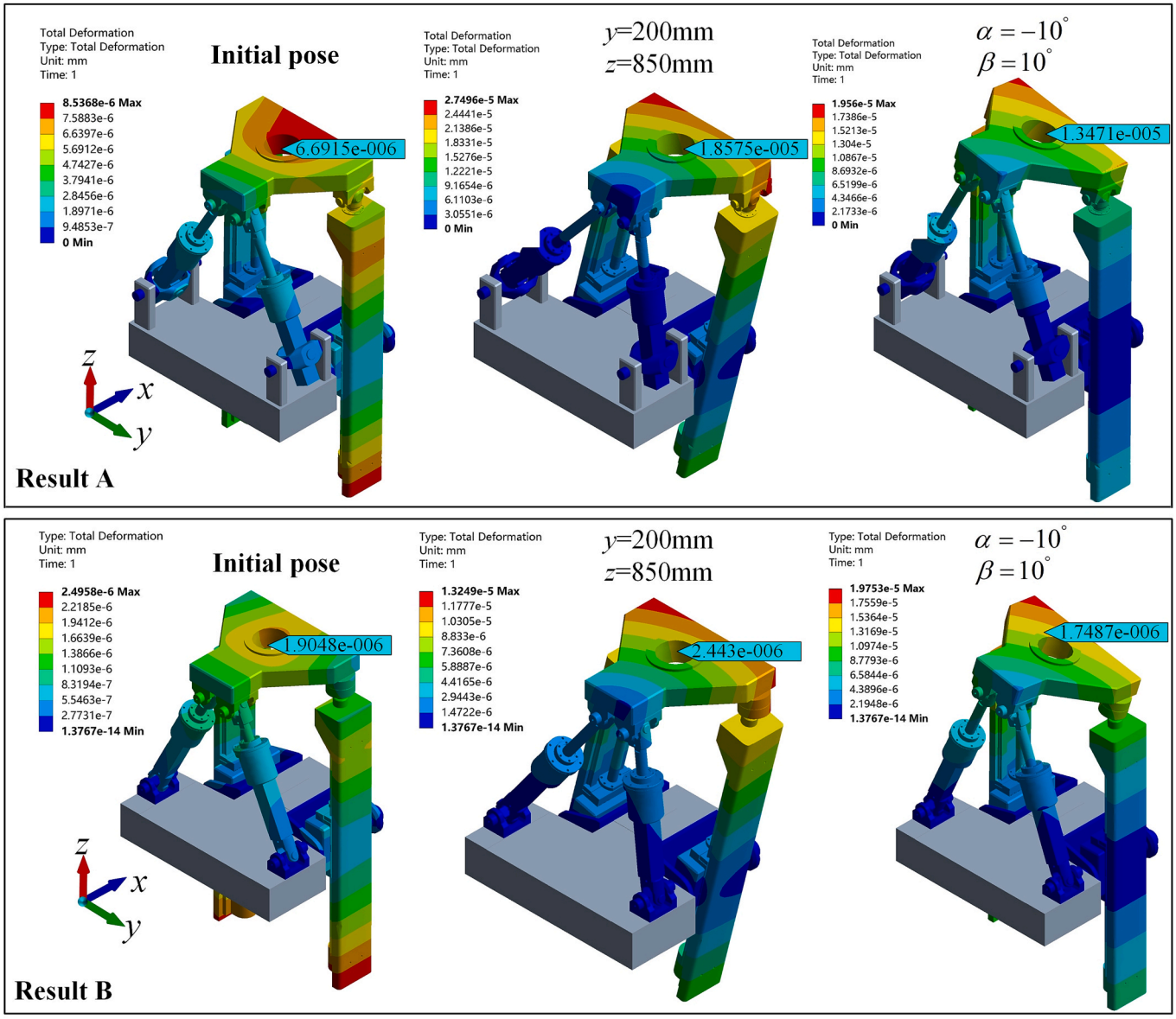


Fig. 14. Stiffness analysis of virtual prototypes A and B.

compared to mechanism B, as illustrated in the "Position Space" and "Attitude Space" results in Figs. 13(a) and (b). The workspace disparity between Virtual Prototypes A and B originates from their distinct U-joint and S-joint configurations. Specifically, prototype A's U-joints and S-joints achieve larger maximum rotational angles than those in prototype B.

For stiffness analysis, three poses of moving platform are selected under a unit force in the Z direction: initial pose, translational pose at $y = 200$mm and $z = 850$mm, rotational pose at $\alpha = -10^{\circ}$, $\beta = 10^{\circ}$. Based on the stiffness analysis results in Fig. 14, the overall stiffness of design A is significantly greater than that of design B. Taking the initial pose as example, the maximum deformation of design B is $2.4958*10^{-6}$mm, while that of design A is $8.1857*10^{-6}$mm.

The ability of the intelligent design system to generate these distinctly different virtual prototypes from the same mechanism topology, each optimized for specific performance requirements, demonstrates the effectiveness and reliability of the proposed design methodology. This successful adaptation to varying design criteria validates both the design approach and the implementation of the intelligent design system.

4.3.2. *Prospective applications*

In industrial robot design, commercial CAD software offers robust capabilities for concept development, modeling, and simulation. Nevertheless, designers still face significant challenges: they must acquire specialized knowledge in mathematics, mechanism theory, and historical design precedents. Gaining the necessary experience and knowledge requires significant time.

To address these limitations, the graph-based representation method for robotic data in this paper enables computers to recognize, store, compute, and apply design-related information, thereby augmenting commercial CAD systems with knowledge support. Although case studies in this work primarily focus on parallel mechanisms (as seen in Fig. 12), the framework accommodates major industrial robot topologies: serial mechanisms functioning as subsystems within parallel structures, parallel mechanisms composed of multiple serial chains sharing a common base and end-effector, and hybrid architectures combining both types [22].

The knowledge support would accelerate design through two key functions:

(1) Automated Fundamental Analyses.

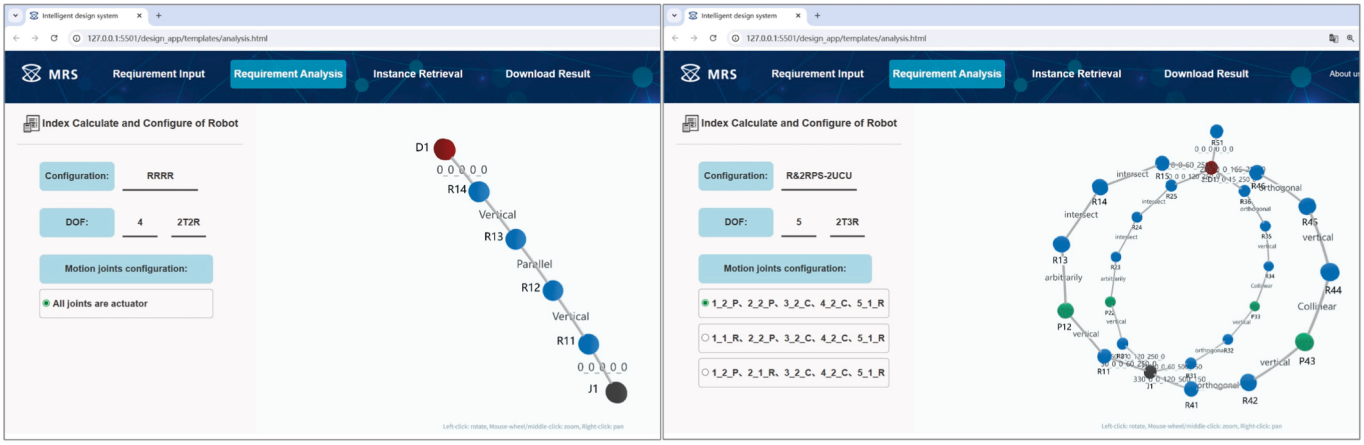


Fig. 15. DoF analysis of (a) serial mechanism and (b) hybrid mechanism.

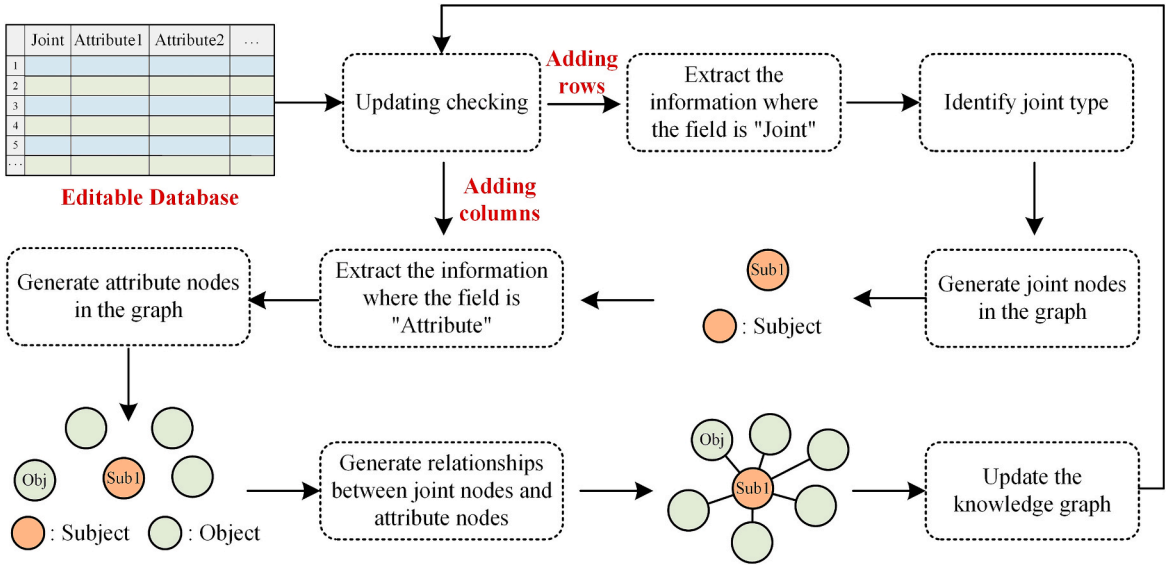


Fig. 16. Updating procedure of knowledge graph.

Industrial robot design encompasses diverse application scenarios. But DoF analysis, constraint analysis, and actuation selection addressed in this paper are basic robotic design steps. Our method helps designers obtain these analyses automatically. When designers input a T-graph, the system can directly read the mechanism's composition and DoF. This reduces designers' dependence on specialized knowledge for these tasks. As shown in Fig. 15, the DoF analysis of serial and hybrid mechanisms are conducted in the system.

Extensibility is enabled through modular reasoning engines. New domain rules (e.g., for emerging technologies) can be integrated by constructing additional reasoning engines. These engines leverage the core computational frameworks established in this paper, including DoF analysis, actuation selection, and motion/constraint analysis. They are all represented as standardized graph structures. Crucially, new engines operate through.json files that define their logic, enabling deployment without modifying the core system architecture.

(2) Facilitated Design Reuse.

Continuous expansion of the component instance library is essential for sustained design innovation. New component instances are added through structured templates as schemed in Fig. 16: adding a new instance appends a row, while introducing a new property inserts a column. Validated instances saved in the designated database are automatically extracted, converted to graph structures, and integrated into the knowledge base during runtime initialization. This enables continuous enrichment from field-proven components.

Through this approach, joint components or sub-assemblies validated in successful designs can be stored as standardized graph representations within the system. This creates a reusable knowledge repository. Engineers across teams can efficiently search, retrieve, and directly integrate these proven components into new designs, avoiding redundant development effort and accelerating the design cycle.

5. Conclusions

This paper focuses on the intelligent framework for component design of industrial robot, which bridges the gap between conceptual specifications and CAD model. The conclusion would be summarized as:

(1) A unified data representation method is proposed on graphs, which effectively captures the intricate interactions between topology, motion/force, component instance, and assembly information in robot design. The system facilitates seamless

information transfer between design stages while enabling automated reasoning and systematic design space exploration, thereby accelerating the overall design process.

(2) The development encompasses three specialized engines: a motion/force analysis engine that combines RBR and CBR techniques, a component instance recommendation engine built on knowledge graphs with embedded domain expertise, and an automated CAD generator utilizing component specifications and interface requirements. These engines coordinate fragmented knowledge to assist designers, relieving them from repetitive and labor-intensive tasks.

(3) The effectiveness of this framework has been demonstrated through the successful design of a parallel machining robot under different performance requirements. The system generated distinct configurations optimized for workspace and stiffness, validating its ability to adapt to varying design objectives.

The framework established in this work provides a solid foundation for further development of automated industrial robot design systems. Future research could focus on expanding the knowledge base to cover a broader range of robot types and integrating additional performance metrics into the design process.

CRediT authorship contribution statement

Tao Sun: Project administration, Funding acquisition, Conceptualization. **Bo Wang:** Writing – original draft, Visualization, Validation, Software. **Xinming Huo:** Writing – review & editing, Methodology, Formal analysis, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported in part by National Natural Science Foundation of China (NSFC) under grant No. 52275028, and in part by Tianjin Outstanding Young Scholar Project under grant No. 22JCJQJC00050, and in part by Tianjin Foundation on Applied Science under grant No. 22JCZDJC00350 and No. 22JCYBJC01670.

Appendix A

The following table shows the pseudo-code for the detailed reasoning process of the joint axes and the calculation of DoF.

Algorithm 1. The reasoning process of joint axis and the calculation of degree of freedom

Input: T-Graph of robot.
Output: DOF of robot.

begin
Extract the number of limbs n, the type of joints, the number of joints m, and the relationship list $C = \{C_1, C_2, \ldots C_{m+1}\}$ from topological graph of robot;
$\mathbf{X} = (n \times m \times 6)$; // Initialize the matrix of axis characteristics
$\mathbf{A} = (k \times 6)$; // Initialize the constraint wrench screw system of robot
// Step1. Loop through the limbs
for ($i = 1$ to n) **do**
  //Step2. Loop through the relationship list and reason the axis characteristics of each joint
  for ($j = 1$ to $m + 1$) **do**
    if (type(C_j) = six-tuple) **then**
      Compute the axes characteristics of initial and terminal joint according to Eqs. (2) and (3);
    else
      Match the rules in Table 2 according to C_j and execute the corresponding conclusions;
    end if
      Update X with the reasoning results of axes characteristics;
  end for
  Compare and integrate the axes characteristics from the two reasonings and update X;
  // Step3. Compute the constraint wrench screw system of robot
  Compute twist of pairs and twist system of limbs according to Eqs. (4) and (5);
  Compute constraint wrench screw of limbs according to Eq. (6) and update A;
end for
Compute twist system of robot according to Eq. (6);
DOF ← rank(twist system of robot)
return DOF;
end

Data availability

Data will be made available on request.

References

[1] D. Mukherjee, K. Gupta, L.H. Chang, H. Najjaran, A survey of robot learning strategies for human-robot collaboration in industrial settings, Robot. Comput. Integr. Manuf. 73 (2022), https://doi.org/10.1016/j.rcim.2021.102231.

[2] H. Jahanshahi, Z.H. Zhu, Review of machine learning in robotic grasping control in space application, Acta Astronaut. 220 (2024) 37–61, https://doi.org/10.1016/j.actaastro.2024.04.012.

[3] Y. Cao, Q. Zhou, W. Yuan, Q. Ye, D. Popa, Y. Zhang, Human-robot collaborative assembly and welding: a review and analysis of the state of the art, J. Manuf. Process. 131 (2024) 1388–1403, https://doi.org/10.1016/j.jmapro.2024.09.044.

[4] K. Chen, M. Wang, X. Huo, P. Wang, T. Sun, Topology and dimension synchronous optimization design of 5-DoF parallel robots for in-situ machining of large-scale steel components, Mech. Mach. Theory 179 (2023), https://doi.org/10.1016/j.mechmachtheory.2022.105105.

[5] C. Dong, H. Liu, W. Yue, T. Huang, Stiffness modeling and analysis of a novel 5-DOF hybrid robot, Mech. Mach. Theory 125 (2018) 80–93, https://doi.org/10.1016/j.mechmachtheory.2017.12.009.

[6] J. Shi, Y. Wang, G. Zhang, H. Ding, Optimal design of 3-DOF PKM module for friction stir welding, Int. J. Adv. Manuf. Technol. 66 (2013) 1879–1889, https://doi.org/10.1007/s00170-012-4467-7.

[7] K. Chen, R. Wang, Z. Niu, P. Wang, T. Sun, Topology design and performance optimization of six-limbs 5-DOF parallel machining robots, Mech. Mach. Theory 185 (2023), https://doi.org/10.1016/j.mechmachtheory.2023.105333.

[8] J. Brinker, B. Corves, Y. Takeda, Kinematic performance evaluation of high-speed Delta parallel robots based on motion/force transmission indices, Mech. Mach. Theory 125 (2018) 111–125, https://doi.org/10.1016/j.mechmachtheory.2017.11.029.

[9] M. Wang, Y. Song, B. Lian, P. Wang, K. Chen, T. Sun, Dimensional parameters and structural topology integrated design method of a planar 5R parallel machining robot, Mech. Mach. Theory 175 (2022), https://doi.org/10.1016/j.mechmachtheory.2022.104964.

[10] T. Huang, M. Li, X.M. Zhao, J.P. Mei, D.G. Chetwynd, S.J. Hu, Conceptual design and dimensional synthesis for a 3-DOF module of the TriVariant - a novel 5-DOF reconfigurable hybrid robot, IEEE Trans. Robot. 21 (3) (2005) 449–456, https://doi.org/10.1109/TRO.2004.840908.

[11] H. Wu, P. Lin, Y. Ma, B. Li, Q. Liu, H. Li, Automated creation of topological structure models of parallel mechanisms for kinematic performance evaluation, IEEE Int. Conf. Adv. Robot. Mechatronics (ICARM) (2023) 1143–1149, https://doi.org/10.1109/ICARM58088.2023.10218779.

[12] T.N. Canh, S.T. Duc, H.N. The, T.H. Dao, X. HoangVan, Optimal design and fabrication of frame structure for dual-arm service robots: an effective approach for human–robot interaction, Eng. Sci. Technol. 56 (2024), https://doi.org/10.1016/j.jestch.2024.101763.

[13] Y. Sun, G. Peng, K. Jin, S. Liu, P. Gardoni, Z. Li, Force/motion transmissibility analysis and parameters optimization of hybrid mechanisms with prescribed workspace, Eng. Anal. Bound. Elem. 139 (2022) 264–277, https://doi.org/10.1016/j.enganabound.2022.03.027.

[14] H. Liu, T. Huang, A. Kecskeméthy, D.G. Chetwynd, Q. Li, Force/motion transmissibility analyses of redundantly actuated and overconstrained parallel manipulators, Mech. Mach. Theory 109 (2017) 126–138, https://doi.org/10.1016/j.mechmachtheory.2016.11.011.

[15] N. Ye, B. Hu, Stiffness modeling of some 4-DOF over-constrained parallel manipulators with various constrained wrench forms, Mech. Mach. Theory 172 (2022), https://doi.org/10.1016/j.mechmachtheory.2022.104821.

[16] J.S. Zhao, S.T. Wei, H.L. Sun, Kinematics and Statics of a 3-UPU robot in screw coordinates, J. Mech. Robot. 15 (6) (2023), https://doi.org/10.1115/1.4056237.

[17] S.H. Kim, B.K. Min, Joint Compliance Error Compensation for Robot Manipulator using Body Frame, Int. J. Precis. Eng. Manuf. 21 (2020) 1017–1023, https://doi.org/10.1007/s12541-019-00300-z.

[18] B. Hu, Kinematically identical manipulators for the Exechon parallel manipulator and their comparison study, Mech. Mach. Theory 103 (2016) 117–137, https://doi.org/10.1016/j.mechmachtheory.2016.05.001.

[19] X. Yang, H.T. Liu, J.L. Xiao, W.L. Zhu, Q. Liu, G.F. Gong, T. Huang, Continuous friction feedforward sliding mode controller for a TriMule hybrid robot, IEEE-ASME T. Mech. 23 (4) (2018) 1673–1683, https://doi.org/10.1109/TMECH.2018.2853764.

[20] Y. Wan, Y. Liu, Z. Chen, C. Chen, X. Li, F. Hu, M. Packianather, Making knowledge graphs work for smart manufacturing: Research topics, applications and prospects, J. Manuf. Syst. 76 (2024) 103–132, https://doi.org/10.1016/j.jmsy.2024.07.009.

[21] J. Li, G. Wu, H. Shen, A. Muller, A. Liu, T.L. Yang, Topology of robotic mechanisms: Framework and mathematics methods – In conjunction with a review of four original theories, Mech. Mach. Theory 175 (2022), https://doi.org/10.1016/j.mechmachtheory.2022.104895.

[22] T. Sun, S. Yang, B. Lian, Finite and Instantaneous Screw Theory in Robotic Mechanism, Springer, Singapore, 2020.

[23] F. Freudenstein, L. Dobrjanskyj, On a theory of type synthesis of mechanisms, Springer, Berlin Heidelberg, 1964, pp. 420–428.

[24] F. Freudenstein, The basic concepts of Polya's theory of enumeration with applications to structural classification of mechanisms, J. Mechanisms 2 (3) (1967) 275–290, https://doi.org/10.1016/0022-2569(67)90003-1.

[25] Y. Lu, Y. Lu, N. Ye, B. Mao, J. Han, C. Sui, Derivation of valid contracted graphs from simpler contracted graphs for type synthesis of closed mechanisms, Mech. Mach. Theory 52 (2012) 206–218, https://doi.org/10.1016/j.mechmachtheory.2012.01.018.

[26] H.S. Yan, C.H. Hsu, Contracted graphs of kinematic chains with multiple joints, Math. Comput. Model. 10 (9) (1988) 681–695, https://doi.org/10.1016/0895-7177(88)90158-6.

[27] E. Santiago-Valenten, E.A. Portilla-Flores, E. Mezura-Montes, E. Vega-Alvarado, M. B. Calva-Yanez, M. Pedroza-Villalba, A graph-theory-based method for topological and dimensional representation of planar mechanisms as a computational tool for engineering design, IEEE Access 7 (2019) 587–596, https://doi.org/10.1109/ACCESS.2018.2885563.

[28] T.S. Mruthyunjaya, Kinematic structure of mechanisms revisited, Mech. Mach. Theory 34 (7) (1999), https://doi.org/10.1016/S0094-114X(02)00120-9.

[29] R.D. Sriram, Intelligent Systems for Engineering: a Knowledge-based Approach, Springer, Verlag, 1997.

[30] J. Girodon, D. Monticolo, E. Bonjour, M. Perrier, An organizational approach to designing an intelligent knowledge-based system: Application to the decision-making process in design projects, Adv. Eng. Inf. 29 (3) (2015) 696–713, https://doi.org/10.1016/j.aei.2015.07.001.

[31] H. Wang, R. Niu, Knowledge service technology for supporting intelligent product design, Shock Vib. 2021 (2021), https://doi.org/10.1155/2021/2561950.

[32] L. Zhang, A. Lobov, Semantic web rule language-based approach for implementing knowledge-based engineering systems, Adv. Eng. Inf. 62 (2024), https://doi.org/10.1016/j.aei.2024.102587.

[33] H. Qin, J. Jin, Y. Zhang, S. Hu, Intelligent design based on knowledge capture and reuse, IOP Conf Ser. Mater. Sci. Eng. 831 (1) (2020), https://doi.org/10.1088/1757-899X/831/1/012008.

[34] B.T. Lin, S.H. Hsu, Automated design system for drawing dies, Expert Syst. Appl. 34 (3) (2008) 1586–1598, https://doi.org/10.1016/j.eswa.2007.01.024.

[35] B. Wang, Y. Song, H. Zhang, J. Wu, Research and application of rule-driven configurable design method for mechanical products, IEEE Int. Conf. Adv. Robot. Mechatronics (ICARM) (2021) 346–351, https://doi.org/10.1109/ICARM52023.2021.9536142.

[36] X. Wei, H. Yuan, H. Wang, Y. Chen, Intelligent design for automotive interior trim structures based on knowledge rule-based reasoning, Int. J. Automot. Technol. 21 (2020) 1149–1167, https://doi.org/10.1007/s12239-020-0109-5.

[37] J. Hu, J. Qi, Y. Peng, New CBR adaptation method combining with problem-solution relational analysis for mechanical design, Comput. Ind. 66 (2015) 41–51, https://doi.org/10.1016/j.compind.2014.08.004.

[38] Y. Huo, J. Liu, J. Xiong, W. Xiao, J. Zhao, Machine learning and CBR integrated mechanical product design approach, Adv. Eng. Inf. 52 (2022), https://doi.org/10.1016/j.aei.2022.101611.

[39] C. Ke, Z. Jiang, H. Zhang, Y. Wang, S. Zhu, An intelligent design for remanufacturing method based on vector space model and case-based reasoning, J. Clean. Prod. 277 (2020), https://doi.org/10.1016/j.jclepro.2020.123269.

[40] M. Chen, R. Qu, W. Fang, Case-based reasoning system for fault diagnosis of aero-engines, Expert Syst. Appl. 202 (2022), https://doi.org/10.1016/j.eswa.2022.117350.

[41] W. Meng, H. Zhang, Q. Ai, T. Bao, J. Yan, CBR-RBR fusion based parametric rapid construction method of bridge BIM model, Adv. Eng. Inf. 57 (2023), https://doi.org/10.1016/j.aei.2023.102086.

[42] X. Long, H. Li, Y. Du, E. Mao, J. Tai, A knowledge-based automated design system for mechanical products based on a general knowledge framework, Expert Syst. Appl. 178 (2021), https://doi.org/10.1016/j.eswa.2021.114960.

[43] G. Peng, G. Chen, C. Wu, H. Xin, Y. Jiang, Applying RBR and CBR to develop a VR based integrated system for machining fixture design, Expert Syst. Appl. 38 (2011) 26–38, https://doi.org/10.1016/j.eswa.2010.05.084.

[44] X. Chen, S. Jia, Y. Xiang, A review: Knowledge reasoning over knowledge graph, Expert Syst. Appl. 141 (2020), https://doi.org/10.1016/j.eswa.2019.112948.

[45] Y. Cheng, Y. Li, N. Zhang, L. Chen, J. Cao, A knowledge graph-enabled multi-domain mapping approach supporting product rapid design: a case study of new energy vehicles, Adv. Eng. Inf. 62 (2024), https://doi.org/10.1016/j.aei.2024.102779.

[46] A.G. Bharadwaj, B. Starly, Knowledge graph construction for product designs from large CAD model repositories, Adv. Eng. Inf. 53 (2022), https://doi.org/10.1016/j.aei.2022.101680.

[47] Y. Guo, L. Wang, Z. Zhang, J. Cao, X. Xia, Y. Liu, Integrated modeling for retired mechanical product genes in remanufacturing: a knowledge graph-based approach, Adv. Eng. Inf. 59 (2024), https://doi.org/10.1016/j.aei.2023.102254.

[48] L. Wang, Y. Fang, D. Zhang, Design of 4-DOF hybrid parallel robots with an integrated three-fingered robot end effector, Mech. Mach. Theory 189 (2023), https://doi.org/10.1016/j.mechmachtheory.2023.105443.

[49] M. Russo, D. Zhang, X.J. Liu, Z. Xie, A review of parallel kinematic machine tools: Design, modeling, and applications, Int. J. Mach. Tools Manuf 196 (2024), https://doi.org/10.1016/j.ijmachtools.2024.104118.

[50] R. Lin, W. Guo, S.S. Cheng, Type synthesis of 2R1T remote center of motion parallel mechanisms with a passive limb for minimally invasive surgical robot, Mech. Mach. Theory 172 (2022), https://doi.org/10.1016/j.mechmachtheory.2022.104766.

[51] A. Wolf, M. Shoham, Screw theory tools for the synthesis of the geometry of a parallel robot for a given instantaneous task, Mech. Mach. Theory 41 (6) (2006) 656–670, https://doi.org/10.1016/j.mechmachtheory.2005.09.006.

[52] X. Huo, T. Sun, Y. Song, A geometric algebra approach to determine motion/constraint, mobility and singularity of parallel mechanism, Mech. Mach. Theory 116 (2017) 273–293, https://doi.org/10.1016/j.mechmachtheory.2017.06.005.

[53] Q. Guo, F. Zhuang, C. Qin, H. Zhu, X. Xie, H. Xiong, Q. He, A survey on knowledge graph-based recommender systems : Extended abstract, IEEE Trans. Knowl. Data Eng. 34 (8) (2022) 3549–3568, https://doi.org/10.1109/TKDE.2020.3028705.

[54] B. Fang, L. Tian, The research on automatic recognition of assembly feature and converse parametric design method based on UG, Adv. Mater. Res. 580 (2012) 87–90, https://doi.org/10.4028/www.scientific.net/AMR.580.87.

[55] A. Neb, J. Hitzer, Automatic generation of assembly graphs based on 3D models and assembly features, Procedia CIRP 88 (2020) 70–75, https://doi.org/10.1016/j.procir.2020.05.013.

[56] X. Zhu, Z. Liu, C. Cai, M. Yang, H. Zhang, L. Fu, J. Zhang, Deep learning-based predicting and compensating method for the pose deviations of parallel robots, Comput. Ind. Eng. 191 (2024), https://doi.org/10.1016/j.cie.2024.110179.

[57] D. Liang, Y.M. Song, T. Sun, G. Dong, Optimum design of a novel redundantly actuated parallel manipulator with multiple actuation modes for high kinematic and dynamic performance, Nonlinear Dyn. 83 (1–2) (2016) 631–658, https://doi.org/10.1007/s11071-015-2353-1.